SIMATS ENGINEERING

Department of Computer Science and Engineering

DATA STRUCTURES

ASSIGNMENT 2 – CO2

Linear Data Structures, Searching, Suffix Arrays and LCP

CO2: Identify the advanced data structures such as balanced search trees, hash tables, priority Queues.

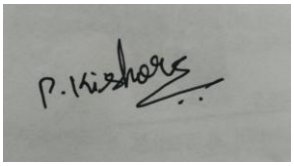
Student Name	Kishore P	**Register Number**	192511046
Department / Section	Computer Science and Engineering	**Academic Year / Semester**	Sep–Dec 2026
Faculty Name	Dr. Supriya L P	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	06-11-2026	**Date of Evaluation**	

Mark Sheet (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Singly Linked Lists	6		Working append / remove / reverse, pointer diagrams after every operation, predicted output, array vs list cost table, defended design for "jump to 500th song"
2	Doubly and Circular Linked Lists	4		List type chosen and justified for each problem, working implementation, node-size and delete-cost comparison, two-stack alternative
3	Stacks	6		Array stack, bracket checker, infix-to-postfix, stack traces for two formulas, overflow / underflow, array vs linked stack
4	Queues	6		Why the linear queue fails, circular queue with modulo, full / empty rule, wrap-around tests, priority extension

Task	Task Title	Max	Marks Awarded	Remarks
5	Linear Search versus Binary Search	7		Both searches, best / average / worst table, measured growth for n = 1,000 / 100,000 / 300,000, break-even calculation
6	Suffix Arrays and LCP Arrays	8		Suffix array, Kasai LCP, banana verified by hand, repeated-substring search, measured growth vs brute force
7	Diagnose, Redesign and Validate	13		Diagnosis, objective, redesign, prediction, original-vs-redesign measurements, defence
	TOTAL	**50**		**Percentage: _____**

OVERALL REMARKS



Student Signature Faculty Signature Program Director

Execution Environment (used for every output in this script)

Item	Value
Compiler	gcc 13.3.0 (Ubuntu 24.04)
Compile command	gcc -std=c11 -Wall -Wextra -pedantic file.c -o program (no warnings)
Memory-safety check	gcc -std=c11 -g -fsanitize=address,undefined file.c -o program_asan (no errors, no leaks)
Timing	the same compile command plus -O2; timing with clock_gettime(CLOCK_MONOTONIC); best of 3 runs
Machine	Intel Xeon @ 2.10 GHz, 1 core, Linux 6.18 (Ubuntu 24.04.5 LTS), cloud container
Random numbers	xorshift64 with a fixed seed, so every run uses the same data

All console outputs below were captured from real runs on this machine. Timings change from run to run and from machine to machine (on this shared single-core container, repeated runs differed by 10–40 % for the sub-millisecond timings); re-run the timing programs on your own computer and replace the timing tables before submitting. The comparison counts, step counts, byte sizes and all non-timing outputs do not change.

Mandatory Submission Evidence Check

✓ = present · ✗ = missing / not satisfied · P = partly present. Based on the "Evidence Expected" of each question. Screenshots and the GitHub link must be added by the student before submission.

Task	Evidence expected	Status	Note
1	Working linked list, pointer diagrams after every operation, cost comparison with the array version, defended design	✓	Diagrams for every operation, measured cost tables, dynamic array chosen for "jump to 500th song" and defended
2	Justified list type for each problem, working implementation, quantified comparison	✓	Doubly linked (history) and circular singly linked (round robin) both implemented; 16 vs 24 bytes and 99,998 vs 1 delete steps measured
3	Working stack with bracket checking and infix-to-postfix, traced stack contents, reasoned complexity	✓	Traces for four formulas and four infix expressions (one error case); overflow / underflow demonstrated; array vs linked stack measured
4	Explained failure, correct circular queue, wrap-around test evidence, reasoned extension	✓	Linear-queue false "full" shown; 17 tokens wrapped through 4 cells with 0 order errors; priority queue and linked queue added
5	Two working searches, comparison-count table, measured growth, break-even calculation	✓	n = 1,000 / 100,000 / 300,000 measured; break-even 133 to 302 look-ups by time, 13 to 24 by comparison count
6	Correct suffix and LCP arrays, repeated-substring search, hand verification, measured growth	✓	banana verified by hand; Kasai trace table; brute force 21.8 s against 25.7 ms at n = 100,000
7	Diagnosis, measurable objective, redesign, prediction, original-vs-redesign evidence, defence	✓	Linear search diagnosed; sorted + binary and hash both validated with 0 disagreements

Implementation Evidence Chain (Q1–Q7)

Evidence item	Status	Observation
Source code	P	Every program is listed in this script and in the separate Program Listing document / zip (Task01 to Task07); the student must commit it to GitHub
Test inputs	✓	Inputs are written inside each program (fixed seed for random data) and shown with the output
Expected behaviour	✓	Expected outputs are derived by hand (trace tables) and compared with the actual output
Actual output	P	Console output is pasted for every program; terminal screenshots must be added by the student

Evidence item	Status	Observation
Intermediate algorithmic evidence	✓	Pointer diagrams, stack traces, dry runs, lo / hi / mid tables and the Kasai trace are included
Boundary / failure tests	✓	Remove of a missing song, full and empty queue, stack overflow and underflow, absent search keys, unbalanced brackets, absent patterns
Interpretation	✓	Each measured table is followed by an explanation against the theoretical growth
Code, data and report consistent	✓	All numbers in the report come from the programs shown; timings are from one machine and should be re-run

Submission Checklist (not separately marked)

Requirement	Status	Note
All 7 questions (Q1–Q7) answered	☒ Yes ☐ Partly ☐ No	The mark sheet lists all seven
Linked lists (Q1–Q2)	☒ Yes ☐ Partly ☐ No	Singly, doubly and circular lists with diagrams and measurements
Stacks (Q3)	☒ Yes ☐ Partly ☐ No	Bracket check, postfix conversion, overflow and underflow
Queues (Q4)	☒ Yes ☐ Partly ☐ No	Circular queue, wrap-around tests, priority extension, linked queue
Searching (Q5)	☒ Yes ☐ Partly ☐ No	Comparison table, growth table, break-even, hashing and interpolation
Suffix and LCP arrays (Q6)	☒ Yes ☐ Partly ☐ No	Hand check, Kasai trace, growth against brute force
Design challenge (Q7)	☒ Yes ☐ Partly ☐ No	Parts A to F answered with measurements
Predictions written before running	☒ Yes ☐ Partly ☐ No	Predictions are derived by hand in a trace table before each run; the student should be able to repeat them in the viva
Every timing table measured on the student's own machine	☐ Yes ☒ Partly ☐ No	Tables here are from a cloud container; replace them with your own
GitHub repository with one commit per task	☐ Yes ☒ Partly ☐ No	To be created by the student; link to be added

DATA STRUCTURES

ASSIGNMENT 2 – CO2

CO2: Identify the advanced data structures such as balanced search trees, hash tables, priority Queues.

Nature of Assignment : Individual | **Duration :** One Week | **Total Marks :** 50 | **Submission Date :** 06 / 11 / 2026

Assignment Title: Linear Data Structures and Searching (50 Marks)

Challenge. Five everyday systems need the right linear data structure: a music playlist (singly linked list), browser history and round-robin scheduling (doubly and circular lists), a spreadsheet formula checker (stack), a passport-office token system (queue), and a result look-up portal (linear and binary search). A DNA and answer-script repeat finder needs suffix and LCP arrays. For each, predict, implement, measure and analyse the solution, then diagnose one program, redesign it and validate the redesign with evidence.

Q	Assessed area	Marks
Q1	Singly linked lists (music playlist)	6
Q2	Doubly and circular linked lists	4
Q3	Stacks (spreadsheet formula checker)	6
Q4	Queues (token system)	6
Q5	Linear search versus binary search	7
Q6	Suffix arrays and LCP arrays	8
Q7	Diagnose, redesign and validate	13
	Total	**50**

How each type of question is answered in this script

Type	What is done
Predict the Output	The output is derived by hand with a trace table, then the program is run and the two are compared

Type	What is done
Complete the Code	The program is compiled with -Wall -Wextra -pedantic, run, and compared with the expected output
Alternative Concept and Complexity Analysis	The alternative is shown with code or pseudocode and compared in time and space with Big-O notation and with measurements

Assessment verification. Students may be asked to explain any prediction, code, test or result in a short viva.

Q1 – Singly Linked Lists (6 Marks)

Aim. To implement a music playlist as a singly linked list with append, remove and in-place reverse, to draw the list and its pointers after every operation, and to compare the list with the fixed array (50 songs) and with a dynamic array.

1.1 Problem and node design

The old playlist keeps its songs in a fixed array of 50 cells. Removing a song shifts every later song one place to the left, and song number 51 has no cell at all, so it simply disappears. A singly linked list has no size limit, and removing a song changes a single pointer instead of moving many songs.

Each song is one node holding its ID and a pointer to the next node. The whole list is identified by the head pointer. An empty list has head == NULL, and the last node has next == NULL.

```
typedef struct Song { int id; struct Song *next; } Song;


head
 |
 v
[21|*]-->[22|*]-->[23|*]-->[24|*]-->[25|/]     ( / = NULL )
```

1.2 Predict the Output

Program q1_predict.c:

```
/* Task 1 - Predict the Output: head insertion and two unlink operations */
#include <stdio.h>
#include <stdlib.h>


typedef struct Node { int v; struct Node *next; } Node;


Node *push_front(Node *h, int v)
{
   Node *n = malloc(sizeof *n);
   n->v = v;
   n->next = h;
   return n;
}
```

```
int main(void)
{
    Node *h = NULL;
    for (int i = 1; i <= 4; i++) h = push_front(h, i * 5);

    Node *p = h->next;          /* second node          */
    Node *gone = p->next;       /* third node           */
    p->next = gone->next;       /* bypass the third     */
    free(gone);

    Node *old = h;              /* drop the head        */
    h = h->next;
    free(old);

    for (Node *t = h; t; t = t->next) printf("%d ", t->v);
    printf("\n");

    while (h) { Node *n = h->next; free(h); h = n; }
    return 0;
}
```

My predicted output (derived by hand before running): 15 5

Statement	List after the statement (head on the left)	Node freed
push_front(h, 5)	5 → NULL	–
push_front(h, 10)	10 → 5 → NULL	–
push_front(h, 15)	15 → 10 → 5 → NULL	–
push_front(h, 20)	20 → 15 → 10 → 5 → NULL	–
p = h->next; gone = p->next;	p points to 15, gone points to 10	–
p->next = gone->next;	20 → 15 → 5 → NULL (10 is bypassed but still allocated)	–
free(gone);	20 → 15 → 5 → NULL	node 10
old = h; h = h->next; free(old);	15 → 5 → NULL	node 20
print loop	prints 15 5	–
final while loop	NULL	nodes 15 and 5

Four nodes are created and two of them (10 and 20) are freed before printing, so only 15 and 5 remain. Head insertion builds the list in the reverse of the creation order (20, 15, 10, 5).

Actual output (captured in terminal):

```
15 5
```

The actual output equals the prediction. The space after the last number comes from the format "%d "; no difference had to be explained.

1.3 Complete the Code

remove_song walks with prev and cur until the ID is found, unlinks the node, frees it and returns the (possibly new) head. reverse turns every next pointer backwards using prev, cur and next. append walks to the last node because the list keeps only a head pointer.

Program q1_playlist.c:

```
/* Task 1 - Music playlist as a singly linked list: append, remove, reverse */
#include <stdio.h>
#include <stdlib.h>


typedef struct Song {
   int id;
   struct Song *next;
} Song;


/* Append at the end: walks the whole list, O(n) */
Song *append(Song *head, int id)
{
   Song *s = malloc(sizeof *s);
   if (!s) return head;
   s->id = id;
   s->next = NULL;
   if (!head) return s;
   Song *t = head;
   while (t->next) t = t->next;
   t->next = s;
   return head;
}
```

```
/* Remove the song with the given id; returns the (possibly new) head */
Song *remove_song(Song *head, int id)
{
    Song *prev = NULL, *cur = head;
    while (cur && cur->id != id) {      /* find node, remember previous */
        prev = cur;
        cur = cur->next;
    }
    if (!cur) return head;              /* id not present: unchanged    */
    if (!prev) head = cur->next;        /* removing the head node       */
    else prev->next = cur->next;        /* bypass the node              */
    free(cur);
    return head;
}


/* Reverse in place using three pointers: O(n) time, O(1) space */
Song *reverse(Song *head)
{
    Song *prev = NULL, *cur = head, *next = NULL;
    while (cur) {
        next = cur->next;   /* 1. remember the rest of the list */
        cur->next = prev;   /* 2. turn the link backwards       */
        prev = cur;         /* 3. move prev forward             */
        cur = next;         /* 4. move cur forward              */
    }
    return prev;            /* prev is the new head             */
}


void print(const Song *h)
{
    for (; h; h = h->next) printf("%d -> ", h->id);
    printf("NULL\n");
}


void free_all(Song *h)
{
    while (h) { Song *n = h->next; free(h); h = n; }
}


int main(void)
{
    Song *pl = NULL;
    for (int id = 21; id <= 25; id++) pl = append(pl, id);
    print(pl);
    pl = remove_song(pl, 21);   /* head removal     */
```

```
    pl = remove_song(pl, 24);   /* middle removal   */
    pl = remove_song(pl, 99);   /* id not present   */
    print(pl);
    pl = reverse(pl);
    print(pl);
    free_all(pl);
    return 0;
}
```

Compiled with gcc -std=c11 -Wall -Wextra -pedantic -g -fsanitize=address,undefined q1_playlist.c -o q1_playlist_asan (no warnings). **Actual output (captured in terminal):**

```
21 -> 22 -> 23 -> 24 -> 25 -> NULL
22 -> 23 -> 25 -> NULL
25 -> 23 -> 22 -> NULL
```

The output matches the expected result: after removing 21 and 24 the list is 22, 23, 25, and after the reverse it is 25, 23, 22. Removing the absent song 99 left the list unchanged. AddressSanitizer printed no error and no leak report, so every node allocated by append was freed exactly once (by remove_song or free_all).

Pointer diagrams after every operation (/ = NULL)

```
append(21)   head-->[21|/]
append(22)   head-->[21|*]-->[22|/]
append(23)   head-->[21|*]-->[22|*]-->[23|/]
append(24)   head-->[21|*]-->[22|*]-->[23|*]-->[24|/]
append(25)   head-->[21|*]-->[22|*]-->[23|*]-->[24|*]-->[25|/]


remove(21)   prev=NULL, cur=21 found at the head: head = 21->next, free(21)
             head-->[22|*]-->[23|*]-->[24|*]-->[25|/]
remove(24)   cur walks 22, 23, 24 (prev ends at 23): 23->next = 24->next, free(24)
             head-->[22|*]-->[23|*]-->[25|/]            [24 freed]
remove(99)   cur walks 22, 23, 25, NULL: not found, list unchanged
             head-->[22|*]-->[23|*]-->[25|/]
reverse()    head-->[25|*]-->[23|*]-->[22|/]
```

Reverse of the list 22, 23, 25: prev, cur and next after each step

Step	prev	cur	next	Links after the step
start	NULL	22	–	22→23→25→NULL
iteration 1	22	23	23	NULL←22 23→25→NULL (22 now points to NULL)
iteration 2	23	25	25	NULL←22←23 25→NULL (23 now points to 22)
iteration 3	25	NULL	NULL	NULL←22←23←25 (25 now points to 23)
return	–	–	–	head = prev = 25, list 25→23→22→NULL

The `next` pointer must be saved first (step 1 of the loop body). If `cur->next = prev` ran before the save, the rest of the list would be lost.

1.4 Inquiry: cost of each operation

Let n be the number of songs and k the position of a song.

Operation	Fixed array (50)	Singly linked list, head only	Singly linked list + tail pointer	Dynamic array (realloc)
Append at the end	O(1), but fails at song 51	O(n): walk to the last node	O(1)	O(1) amortised, O(n) on the one resize
Remove a song by ID	O(n): find, then shift n − k left	O(n) to find, O(1) to unlink	O(n) to find, O(1) to unlink	O(n): find, then shift
Reach the k-th song	O(1)	O(k)	O(k)	O(1)
Reverse	O(n) swaps, O(1) space	O(n), O(1) space (three pointers)	O(n), O(1) space	O(n) swaps, O(1) space
Space per song	4 bytes, 200 bytes reserved even if empty	16 bytes (sizeof(Song)) plus allocator overhead	16 bytes plus allocator overhead	4 bytes, up to 2 × the songs reserved after a doubling

Effect of a tail pointer. The list keeps a second pointer `tail` to the last node. Append becomes `s->next = NULL; tail->next = s; tail = s;`: three statements with no loop, so O(1) instead of O(n). The

price is that tail must be kept correct. Removing the last node still needs the node before it (O(n) in a singly linked list), and removing the only node must set tail = NULL.

Measured cost (q1_bench.c, -O2, times in milliseconds, full output below):

```
A. Append n songs (ms)
      n   list,no tail    list,tail     dyn. array
   1000         0.922        0.006          0.005
   5000        24.910        0.028          0.027
  20000       376.612        0.126          0.267


B. 1,000 look-ups of the song at position n/2 (ms)
      n  linked list   dyn. array
   1000       0.785        0.001
   5000       6.457        0.001
  20000      27.352        0.001


C. 1,000 removals around the middle (ms)
      n  linked list   dyn. array
   5000       6.819        0.778
  20000      24.098        4.368


sizeof(Song)=16 bytes, sizeof(int)=4 bytes
```

n	Append n songs: list, no tail	list, tail	dynamic array
1,000	0.922	0.006	0.005
5,000	24.910	0.028	0.027
20,000	376.612	0.126	0.267

Without a tail, the cost grows about 15 times when n grows 4 times (5,000 to 20,000: 24.910 to 376.612 ms), which is the signature of $O(n^2)$ for n appends. With a tail it grows about 4.5 times (0.028 to 0.126 ms), which is O(n) for n appends, that is O(1) per append. The dynamic array grew about 10 times here; at these sub-millisecond times the realloc calls and timer noise dominate, but the absolute cost is still a thousand times below the list without a tail.

n	1,000 look-ups of the song at position n/2: linked list	dynamic array
1,000	0.785	0.001
5,000	6.457	0.001
20,000	27.352	0.001

The list time grows with k (O(k)): 6.457 to 27.352 ms is 4.2 times for 4 times the position. The array time does not change (O(1)); 0.001 ms for 1,000 look-ups is at the resolution of the timer.

n	1,000 removals near the middle: linked list	dynamic array (memmove)
5,000	6.819	0.778
20,000	24.098	4.368

Both are O(n), but the array was 5 to 9 times faster, because the search and the shift run over contiguous memory while the list search jumps between nodes scattered in memory. (n = 1,000 is left out: the 1,000 removals would empty the whole list, so the test would not be comparable.) So "O(1) unlink" only helps when the node is already in hand.

1.5 Design Decision: jump to the 500th song

Decision: keep the playlist in a dynamic array of song IDs, not in a singly linked list.

- Jumping to song 500 is `a[499]`, O(1). In the list it needs 499 pointer hops, O(k). The measurement above (1,000 look-ups at position 500 in 0.785 ms) puts one jump at about 0.8 microseconds, so a single jump is not slow, but the cost grows with the position and with every user who skips around.
- Append stays O(1) amortised, the 50-song limit disappears (the capacity doubles), and the array stores 4 bytes per song instead of 16.
- Removal is O(n) in both versions, and in the measurement the array was faster.
- In-place reverse is still O(n) time and O(1) space for the array (swap `a[i]` with `a[len-1-i]`), so the list's reverse is no advantage.

Where the linked list is still the better choice: a playlist with very frequent inserts and removals at a position the program already holds (for example "play next" right after the current song), because no song moves and no pointer is invalidated. A realloc can move the whole array, so any pointer into it becomes invalid, and the one resize is an O(n) pause. A hybrid is possible: an array of IDs plus a current-song index.

1.6 Alternative Concept: dynamic array

```
typedef struct { int *id; int len, cap; } Playlist;


void pl_append(Playlist *p, int id) {          /* amortised O(1) */
   if (p->len == p->cap) {
      p->cap = p->cap ? p->cap * 2 : 1;       /* double the capacity */
      p->id = realloc(p->id, (size_t)p->cap * sizeof *p->id);
   }
   p->id[p->len++] = id;
}
void pl_remove(Playlist *p, int id) {          /* O(n) */
   int i = 0;
   while (i < p->len && p->id[i] != id) i++;    /* find */
   if (i < p->len) {
      memmove(&p->id[i], &p->id[i + 1], (size_t)(p->len - i - 1) * sizeof *p->id);
      p->len--;
   }
}
void pl_reverse(Playlist *p) {                 /* O(n) time, O(1) space */
   for (int i = 0, j = p->len - 1; i < j; i++, j--) {
      int t = p->id[i]; p->id[i] = p->id[j]; p->id[j] = t;
   }
}
```

Measure	Singly linked list	Dynamic array
Append	O(n) without tail, O(1) with tail	O(1) amortised (copies 1 + 2 + 4 + … < 2n elements over n appends)
Remove a song	O(n)	O(n)
k-th song	O(k)	O(1)

Measure	Singly linked list	Dynamic array
Space per song	16 bytes node (4 data + 4 padding + 8 pointer), measured sizeof(Song) = 16, plus one malloc header per node	4 bytes, plus unused capacity (less than 100 % extra)
Cache behaviour	nodes scattered in memory	contiguous, fast scans
Failure modes	malloc failure, leaks if a node is not freed	realloc failure, invalidated pointers after growth

1.7 Viva questions with model answers

1. **Why does `remove_song` return the head?** The head may change (when the first song is removed), and C passes the pointer by value, so the caller must receive the new head.
2. **Why is `next` saved before `cur->next = prev` in reverse?** Otherwise the link to the rest of the list is overwritten and the remaining songs are lost.
3. **What does `remove_song(pl, 99)` do?** cur reaches NULL, the function returns the unchanged head, and no memory is freed.
4. **Why is `append` O(n) here?** There is no tail pointer, so the loop walks to the last node.
5. **What goes wrong if `free(cur)` is called before `prev->next = cur->next`?** cur->next is then read from freed memory, which is undefined behaviour.

1.8 Conclusion

The singly linked list removes the 50-song limit and gives an in-place reverse with three pointers; a tail pointer turns append from O(n) into O(1). The measurements show its weakness: reaching or finding a song costs O(k) with poor cache behaviour (27.352 ms against 0.001 ms for 1,000 look-ups at n = 20,000). For a playlist that must also jump to the 500th song, a dynamic array is the better design, and the list is kept for edits at a known position.

Q2 – Doubly and Circular Linked Lists (4 Marks)

Aim. To choose the right kind of linked list for browser history (Back and Forward) and for round-robin scheduling, to implement both, to compare singly and doubly linked nodes in memory and in delete cost, and to compare the doubly linked list with a two-stack design.

2.1 Choosing the list type

List type	Links per node	Strength	Weakness
Singly	next	smallest node	cannot go backwards; delete needs the previous node
Doubly	prev, next	moves both ways; O(1) delete with a node pointer	extra pointer, two links to update
Circular singly	next; last node points to first	wrap-around without a special case	no backward movement
Circular doubly	prev, next; ends joined	wrap-around both ways	most pointers to maintain

Problem	Needed operations	Choice	Reason
Browser history	Back and Forward from the current page; a new page after the current one	Doubly linked list	Back needs prev, Forward needs next. History has a first and a last page, so no wrap-around is wanted, which rules out the circular types.
Round-robin scheduling	After the last job return to the first; remove a finished job	Circular singly linked list (circular doubly if jobs must also be removed by a pointer or visited backwards)	Time moves in one direction only, and wrap-around is exactly what the circular link gives. One pointer per job is enough.

Both problems are implemented below.

2.2 Implementation A: browser history with a doubly linked list

Program q2_history.c (a three-page visit followed by Back, Back and Forward, extended with a new visit after going Back to show that the Forward pages are discarded):

```
/* Task 2A - Browser history as a doubly linked list (Back / Forward / new visit) */
#include <stdio.h>
#include <stdlib.h>


typedef struct Page {
   char url[100];
   struct Page *prev, *next;
} Page;


/* Visit a new page after 'current'; all old Forward pages are freed */
Page *visit(Page *current, const char *url)
{
   Page *p = malloc(sizeof *p);
   if (!p) return current;
   snprintf(p->url, sizeof p->url, "%s", url);
   p->prev = current;
   p->next = NULL;
   if (current) {
      Page *discard = current->next;      /* old Forward pages */
      current->next = p;
      while (discard) {                   /* free them: Forward is cleared */
         Page *n = discard->next;
         free(discard);
         discard = n;
      }
   }
   return p;
}


Page *back(Page *p)    { return (p && p->prev) ? p->prev : p; }
Page *forward(Page *p) { return (p && p->next) ? p->next : p; }


int main(void)
{
   Page *cur = NULL;
   cur = visit(cur, "login");
   cur = visit(cur, "dashboard");
   cur = visit(cur, "reports");
   cur = back(cur);    printf("Back: %s\n", cur->url);
   cur = back(cur);    printf("Back: %s\n", cur->url);
```

```
    cur = forward(cur); printf("Forward: %s\n", cur->url);
    cur = visit(cur, "settings");   /* 'reports' must be discarded */
    cur = forward(cur); printf("Forward after new visit: %s (no Forward page)\n", cur->url);
    cur = back(cur);    printf("Back: %s\n", cur->url);
    cur = back(cur);    printf("Back: %s\n", cur->url);
    cur = back(cur);    printf("Back at oldest page stays: %s\n", cur->url);


    while (cur && cur->prev) cur = cur->prev;       /* free everything */
    while (cur) { Page *n = cur->next; free(cur); cur = n; }
    return 0;
}
```

Actual output (captured in terminal, compiled with -fsanitize=address,undefined, no errors, no leaks):

```
Back: dashboard
Back: login
Forward: dashboard
Forward after new visit: settings (no Forward page)
Back: dashboard
Back: login
Back at oldest page stays: login
```

The first three lines are the basic Back / Back / Forward behaviour. The last four lines show that after visiting settings the page reports is gone, that forward leaves the current page unchanged when there is no next page, and that back stops at the oldest page.

Pointers after each step (* marks the current page)

```
visit login       NULL <- [login*] -> NULL
visit dashboard   NULL <- [login] <-> [dashboard*] -> NULL
visit reports     NULL <- [login] <-> [dashboard] <-> [reports*] -> NULL
Back              NULL <- [login] <-> [dashboard*] <-> [reports] -> NULL
Back              NULL <- [login*] <-> [dashboard] <-> [reports] -> NULL
Forward           NULL <- [login] <-> [dashboard*] <-> [reports] -> NULL
visit settings    reports freed: NULL <- [login] <-> [dashboard] <-> [settings*] -> NULL
```

2.3 Implementation B: round-robin scheduling with a circular list

The list is kept by its **tail** pointer: tail->next is the first job, so adding at the end and finding the first job are both O(1). Program q2_rr.c runs four jobs with burst times 7, 4, 6 and 3 and a time quantum of 4:

```
/* Task 2B - Round-robin scheduling with a circular singly linked list.
   The list is kept by its TAIL pointer: tail->next is the first job. */
#include <stdio.h>
#include <stdlib.h>


typedef struct Job { int id, remaining; struct Job *next; } Job;


Job *add(Job *tail, int id, int burst)
{
    Job *j = malloc(sizeof *j);
    if (!j) return tail;
    j->id = id;
    j->remaining = burst;
    if (!tail) { j->next = j; return j; }   /* single node points to itself */
    j->next = tail->next;                   /* new node -> first job        */
    tail->next = j;                         /* old tail -> new node         */
    return j;                               /* new tail                     */
}


int main(void)
{
    int burst[] = {7, 4, 6, 3}, quantum = 4, n = 4;
    Job *tail = NULL;
    for (int i = 0; i < n; i++) tail = add(tail, i + 1, burst[i]);


    Job *prev = tail, *cur = tail->next;    /* cur = first job */
    int time = 0;
    while (cur) {
        int run = cur->remaining < quantum ? cur->remaining : quantum;
        printf("t=%2d  job %d runs %d unit(s)", time, cur->id, run);
        time += run;
        cur->remaining -= run;
        if (cur->remaining == 0) {
            printf("  -> finished at t=%d\n", time);
            if (cur->next == cur) { free(cur); break; }     /* last job */
            prev->next = cur->next;                         /* unlink   */
            Job *dead = cur;
            cur = cur->next;
```

```
            free(dead);
        } else {
            printf("  (remaining %d)\n", cur->remaining);
            prev = cur;
            cur = cur->next;              /* wrap-around is automatic */
        }
    }
    return 0;
}
```

Actual output (captured in terminal, AddressSanitizer clean):

```
t= 0  job 1 runs 4 unit(s)  (remaining 3)
t= 4  job 2 runs 4 unit(s)  -> finished at t=8
t= 8  job 3 runs 4 unit(s)  (remaining 2)
t=12  job 4 runs 3 unit(s)  -> finished at t=15
t=15  job 1 runs 3 unit(s)  -> finished at t=18
t=18  job 3 runs 2 unit(s)  -> finished at t=20
```

After job 4 the scheduler returns to job 1 without any special code, because the last node points to the first. The total time 20 = 7 + 4 + 6 + 3, as it must be. A finished job is removed with prev->next = cur->next; this is why the scheduler keeps prev beside cur, since a singly linked node cannot find its predecessor by itself.

Job	Burst	Finish time (turnaround)	Waiting time = turnaround − burst
1	7	18	11
2	4	8	4
3	6	20	14
4	3	15	12
Average		**15.25**	**10.25**

2.4 Analysis: singly versus doubly linked nodes

Program q2_compare.c measures the node sizes and the cost of deleting the last node when only a pointer to it is held. **Actual output (captured in terminal):**

```
sizeof(SN)=16  sizeof(DN)=24  sizeof(SP)=112  sizeof(DP)=120
```

```
Delete the LAST node when only a pointer to it is held (1,000 repetitions)
     n   singly steps   doubly steps      singly ms      doubly ms
  1000            998              1          1.942          0.000
 10000           9998              1         23.139          0.000
100000          99998              1        352.400          0.000
```

Measure	Singly linked	Doubly linked
Node with an int payload (measured sizeof)	16 bytes	24 bytes (+50 %)
Page node with a 100-char URL (measured sizeof)	112 bytes	120 bytes (+7 %)
Delete a node when only a pointer to it is known	O(n): walk from the head to find the predecessor	O(1): n->prev->next = n->next; n->next->prev = n->prev;
Pointers changed by a middle delete	1	2 (plus 2 more at the ends)

The extra pointer costs 8 bytes per node: large relative to a small payload, small relative to a big record.

n	Singly: steps	Doubly: steps	Singly: time (ms)	Doubly: time (ms)
1,000	998	1	1.942	< 0.001
10,000	9,998	1	23.139	< 0.001
100,000	99,998	1	352.400	< 0.001

"Steps" counts the nodes visited in one delete; the time is for 1,000 repetitions. The singly linked cost grows in step with n (O(n)); the doubly linked cost is constant (O(1)) and too small for the timer to show (printed as 0.000 ms).

2.5 Alternative Concept: two stacks for browser history

Keep a Back stack, a Forward stack and a variable current.

- **Visit a new page:** push current onto Back, set current to the new page, empty Forward.

- **Back:** push current onto Forward, pop Back into current.
- **Forward:** push current onto Back, pop Forward into current.

Program q2_twostack.c replays the same visits. **Actual output (captured in terminal):**

```
start                Back=[] Current=login Forward=[]
visit dashboard      Back=[login] Current=dashboard Forward=[]
visit reports        Back=[login,dashboard] Current=reports Forward=[]
Back                 Back=[login] Current=dashboard Forward=[reports]
Back                 Back=[] Current=login Forward=[dashboard,reports]
Forward              Back=[login] Current=dashboard Forward=[reports]
visit settings (clears Fwd) Back=[login,dashboard] Current=settings Forward=[]
```

```
/* Task 2 - Alternative: browser history with a Back stack and a Forward stack */
#include <stdio.h>
#include <string.h>

#define CAP 16
typedef struct { const char *s[CAP]; int top; } Stack;

static void push(Stack *st, const char *u) { if (st->top < CAP - 1) st->s[++st->top] = u; }
static const char *pop(Stack *st) { return st->top >= 0 ? st->s[st->top--] : NULL; }

static void show(const char *what, const Stack *b, const char *cur, const Stack *f)
{
   printf("%-22s Back=[", what);
   for (int i = 0; i <= b->top; i++) printf("%s%s", i ? "," : "", b->s[i]);
   printf("] Current=%s Forward=[", cur);
   for (int i = f->top; i >= 0; i--) printf("%s%s", i != f->top ? "," : "", f->s[i]);
   printf("]\n");
}

int main(void)
{
   Stack back = {.top = -1}, fwd = {.top = -1};
   const char *cur = "login";
   show("start", &back, cur, &fwd);

   push(&back, cur); cur = "dashboard"; fwd.top = -1; show("visit dashboard", &back, cur, &fwd);
   push(&back, cur); cur = "reports";   fwd.top = -1; show("visit reports", &back, cur, &fwd);

   push(&fwd, cur); cur = pop(&back); show("Back", &back, cur, &fwd);
   push(&fwd, cur); cur = pop(&back); show("Back", &back, cur, &fwd);
```

```
    push(&back, cur); cur = pop(&fwd); show("Forward", &back, cur, &fwd);


    push(&back, cur); cur = "settings"; fwd.top = -1; show("visit settings (clears Fwd)", &back, cur, &fwd);
    return 0;
}
```

The same pages appear in the same order as in the doubly linked list, and visiting settings clears the Forward pages exactly as visit freed reports above.

Measure	Doubly linked list	Two stacks
Back, Forward, visit (time)	O(1)	O(1) (array stacks: clearing Forward is top = -1)
Space for h retained pages	O(h), 2 pointers per page (16 bytes)	O(h): each page is stored once, in exactly one stack or as current; array stacks store no pointers but reserve a fixed capacity
Discarding Forward pages	O(f) to free f nodes (each node freed once, so O(1) amortised)	O(1) with an array stack (O(f) with a linked stack)
Jump to the middle of history	O(k) walk	not possible without popping

The doubly linked list is the more flexible choice (it can also show the whole history and delete one entry); the two stacks are simpler and just as fast for the three required operations.

2.6 Viva questions with model answers

1. **Why is a circular list suited to round robin?** After the last job the scheduler must return to the first; the last node already points there, so no check or reset is needed.
2. **Why keep a tail pointer in the circular list?** tail->next gives the first job and tail itself gives the end, so both add-at-end and start-from-first are O(1).
3. **What must visit do with the Forward pages?** Free them; after a new page the old Forward history no longer makes sense, and keeping them would leak memory.
4. **Why does a doubly linked list delete in O(1)?** The node knows its own predecessor, so no search is needed.

2.7 Conclusion

Browser history is a doubly linked list (two directions, no wrap-around) and round robin is a circular singly linked list (one direction, wrap-around). Doubly linked nodes cost 8 more bytes but make delete-by-pointer O(1) instead of O(n), measured as 1 step against 99,998 steps at n = 100,000. Two stacks give the same O(1) navigation for history.

Q3 – Stacks **(6 Marks)**

Aim. To implement a stack as an array, to use it for bracket checking and infix-to-postfix conversion of spreadsheet formulas, to trace the stack contents, to explain overflow and underflow, and to compare the array stack with a linked-list stack.

3.1 Theory

A stack is a Last-In-First-Out (LIFO) structure: the last item pushed is the first item popped, and only the top is visible. The array version keeps the items in item[0..top] and a counter top (−1 when the stack is empty).

Operation	Meaning	Precondition	Time
push(x)	put x on the top	stack not full	O(1)
pop()	remove and return the top	stack not empty	O(1)
is_empty()	top == -1	none	O(1)

Bracket matching and postfix conversion both need "the most recent unfinished thing first", which is exactly LIFO: the latest opening bracket must be closed first, and the latest operator waiting must be handled first.

3.2 Predict the Output

Program q3_predict.c:

```
/* Task 3 - Predict the Output: array stack with a nested pop inside push */
#include <stdio.h>


int st[10], top = -1;
void push(int x) { st[++top] = x; }
int  pop(void)   { return st[top--]; }


int main(void)
{
   push(2); push(4); push(6);
   printf("%d ", pop());
   push(8); push(10);
   printf("%d ", pop());
```

```
    printf("%d ", pop());
    push(pop() * 3);
    while (top >= 0) printf("%d ", pop());
    printf("\n");
    return 0;
}
```

My predicted output (derived by hand before running): 6 10 8 12 2

Statement	Stack after it (bottom → top)	Printed
push(2); push(4); push(6);	[2, 4, 6]	
printf(pop())	[2, 4]	6
push(8); push(10);	[2, 4, 8, 10]	
printf(pop())	[2, 4, 8]	10
printf(pop())	[2, 4]	8
push(pop() * 3): pop returns 4	[2]	
… then push(12)	[2, 12]	
loop: pop	[2]	12
loop: pop	[ ]	2

Actual output (captured in terminal):

```
6 10 8 12 2
```

The actual output equals the prediction (the space after 2 comes from the "%d " format). The inner pop() in push(pop() * 3) runs first and removes 4 before 12 is pushed, so 4 never reaches the final loop.

3.3 Complete the Code: bracket checker

The body of the for loop pushes every opening bracket; for a closing bracket it pops, and fails if the stack was empty (underflow) or the popped bracket is of the wrong type. After the loop the

formula is valid only if the stack is empty (no unmatched opener left). The extra function trace prints the stack after every bracket for the tables below.

Program q3_brackets.c:

```
/* Task 3 - Spreadsheet formula checker: bracket matching with an array stack */
#include <stdio.h>
#include <string.h>


#define MAX 100


typedef struct {
    char item[MAX];
    int top;              /* index of the top item, -1 when empty */
} Stack;


void push(Stack *s, char c) { if (s->top < MAX - 1) s->item[++s->top] = c; }
char pop(Stack *s)          { return s->top >= 0 ? s->item[s->top--] : '\0'; }
int  is_empty(const Stack *s) { return s->top == -1; }


/* Returns 1 if every bracket in the formula is correctly matched */
int balanced(const char *expr)
{
    Stack s = {.top = -1};
    for (size_t i = 0; expr[i]; i++) {
        char c = expr[i];
        if (c == '(' || c == '[' || c == '{') {
            if (s.top == MAX - 1) return 0;        /* overflow guard */
            push(&s, c);
        } else if (c == ')' || c == ']' || c == '}') {
            char open = pop(&s);
            if (open == '\0') return 0;            /* underflow: nothing to match */
            if ((c == ')' && open != '(') ||
                (c == ']' && open != '[') ||
                (c == '}' && open != '{')) return 0; /* wrong bracket type */
        }
    }
    return is_empty(&s);                           /* leftover opener = invalid */
}


/* Same logic, printing the stack after every bracket (for the trace tables) */
void trace(const char *expr)
{
    Stack s = {.top = -1};
```

```
    printf("Formula %s\n", expr);
    for (size_t i = 0; expr[i]; i++) {
        char c = expr[i];
        if (c == '(' || c == '[' || c == '{') {
            push(&s, c);
            printf("  read '%c' (pos %2zu): pushed        ", c, i);
        } else if (c == ')' || c == ']' || c == '}') {
            char open = pop(&s);
            printf("  read '%c' (pos %2zu): popped '%c'     ", c, i, open ? open : '-');
            if (open == '\0') { printf("-> UNDERFLOW, invalid\n"); return; }
            if ((c == ')' && open != '(') || (c == ']' && open != '[') || (c == '}' && open != '{')) {
                printf("-> MISMATCH, invalid\n");
                return;
            }
            printf("-> ok  ");
        } else continue;
        printf("stack(bottom..top) = [");
        for (int k = 0; k <= s.top; k++) printf("%c", s.item[k]);
        printf("]\n");
    }
    printf("  end of formula: stack %s\n", is_empty(&s) ? "empty -> valid" : "NOT empty -> invalid");
}


int main(void)
{
    const char *cells[] = {"=AVERAGE(B2:B9)*{C1+(D4-1)}", "=[A1+(B2*C3])",
                           "=((B2+C2)*D2", "=SUM(A1:A5))"};
    for (int i = 0; i < 4; i++)
        printf("%-30s %s\n", cells[i], balanced(cells[i]) ? "valid" : "invalid");
    printf("\n");
    for (int i = 0; i < 4; i++) { trace(cells[i]); printf("\n"); }
    return 0;
}
```

Actual output (captured in terminal, compiled with -Wall -Wextra -pedantic, no warnings):

```
=AVERAGE(B2:B9)*{C1+(D4-1)}    valid
=[A1+(B2*C3])                  invalid
=((B2+C2)*D2                   invalid
=SUM(A1:A5))                    invalid


Formula =AVERAGE(B2:B9)*{C1+(D4-1)}
  read '(' (pos  8): pushed        stack(bottom..top) = [(]
  read ')' (pos 14): popped '('     -> ok  stack(bottom..top) = []
  read '{' (pos 16): pushed        stack(bottom..top) = [{]
```

```
  read '(' (pos 20): pushed       stack(bottom..top) = [{(]
  read ')' (pos 25): popped '('    -> ok  stack(bottom..top) = [{]
  read '}' (pos 26): popped '{'    -> ok  stack(bottom..top) = []
  end of formula: stack empty -> valid


Formula =[A1+(B2*C3])
  read '[' (pos  1): pushed       stack(bottom..top) = [[]
  read '(' (pos  5): pushed       stack(bottom..top) = [[(]
  read ']' (pos 11): popped '('    -> MISMATCH, invalid


Formula =((B2+C2)*D2
  read '(' (pos  1): pushed       stack(bottom..top) = [(]
  read '(' (pos  2): pushed       stack(bottom..top) = [((]
  read ')' (pos  8): popped '('    -> ok  stack(bottom..top) = [(]
  end of formula: stack NOT empty -> invalid


Formula =SUM(A1:A5))
  read '(' (pos  4): pushed       stack(bottom..top) = [(]
  read ')' (pos 10): popped '('    -> ok  stack(bottom..top) = []
  read ')' (pos 11): popped '-'    -> UNDERFLOW, invalid
```

The four classifications are: the first formula is valid, and the other three are invalid for three different reasons (wrong bracket type, an opener never closed, and a closer with no opener).

Stack for =[A1+(B2*C3]) after each bracket (mismatch case)

Character	Action	Stack (bottom → top)	Result
[(position 1)	push	[	
((position 5)	push	[(	
] (position 11)	pop gives (, but] needs [	[	mismatch found here, formula rejected

The mismatch is found at the] after C3: the most recent open bracket is (, so] cannot close it. The final) is never read.

Stack for the valid formula =AVERAGE(B2:B9)*{C1+(D4-1)}

Character	Action	Stack (bottom → top)
((position 8)	push	(
) (position 14)	pop gives (, matches	(empty)
{ (position 16)	push	{
((position 20)	push	{ (
) (position 25)	pop gives (, matches	{
} (position 26)	pop gives {, matches	(empty)
end of formula	stack empty	valid

3.4 Infix to postfix with a stack

Rules. Operands go straight to the output. (is pushed.) pops operators to the output until (, which is discarded. An operator first pops every stack operator of higher precedence (or equal precedence, for left-associative operators), then is pushed. At the end everything left on the stack is popped. Precedence: ^ (3, right-associative), * / (2), + - (1).

Program q3_postfix.c (single-letter operands; a spreadsheet cell such as A1 would be read as one token by a tokenizer placed in front):

```
/* Task 3 - Infix to postfix conversion with an operator stack.
   Single-letter operands; precedence  ^ (3, right-assoc)  * / (2)  + - (1) */
#include <stdio.h>
#include <ctype.h>


#define MAX 100


typedef struct { char item[MAX]; int top; } Stack;


static int  push(Stack *s, char c)  { if (s->top == MAX - 1) return 0; s->item[++s->top] = c; return 1; }
static int  pop(Stack *s, char *c)  { if (s->top < 0) return 0; *c = s->item[s->top--]; return 1; }
static char peek(const Stack *s)    { return s->top >= 0 ? s->item[s->top] : '\0'; }
static int  prec(char op)
{
   switch (op) {
```

```
    case '^': return 3;
    case '*': case '/': return 2;
    case '+': case '-': return 1;
    default:  return 0;
    }
}

/* returns 1 on success, 0 on unbalanced brackets or stack overflow */
int to_postfix(const char *in, char *out, int verbose)
{
    Stack s = {.top = -1};
    int k = 0;
    char c;
    if (verbose) printf("%-5s %-14s %s\n", "read", "stack", "output");
    for (int i = 0; in[i]; i++) {
        char t = in[i];
        if (isalnum((unsigned char)t)) {
            out[k++] = t;                       /* operand: straight to output */
        } else if (t == '(') {
            if (!push(&s, t)) return 0;
        } else if (t == ')') {
            while (peek(&s) != '(' && pop(&s, &c)) out[k++] = c;
            if (!pop(&s, &c)) return 0;             /* missing '(' */
        } else {                                /* operator */
            while (s.top >= 0 && peek(&s) != '(' &&
                   (prec(peek(&s)) > prec(t) || (prec(peek(&s)) == prec(t) && t != '^'))) {
                pop(&s, &c);
                out[k++] = c;
            }
            if (!push(&s, t)) return 0;
        }
        if (verbose) { out[k] = 0; printf("%-5c [%-12.*s] %s\n", t, s.top + 1, s.item, out); }
    }
    while (pop(&s, &c)) {
        if (c == '(') return 0;                 /* missing ')' */
        out[k++] = c;
    }
    out[k] = 0;
    if (verbose) printf("%-5s [%-12s] %s\n", "end", "", out);
    return 1;
}

int main(void)
{
    const char *tests[] = {"A*B+C", "A+(B-C)*D/E", "A^B^C", "(A+B"};
    char out[MAX];
```

```
    for (int i = 0; i < 4; i++) {
        printf("Infix: %s\n", tests[i]);
        if (to_postfix(tests[i], out, 1)) printf("Postfix: %s\n\n", out);
        else printf("Postfix: error (unbalanced brackets)\n\n");
    }
    return 0;
}
```

Actual output (captured in terminal):

```
Infix: A*B+C
read  stack         output
A     [             ] A
*     [*            ] A
B     [*            ] AB
+     [+            ] AB*
C     [+            ] AB*C
end   [             ] AB*C+
Postfix: AB*C+


Infix: A+(B-C)*D/E
read  stack         output
A     [             ] A
+     [+            ] A
(     [+(           ] A
B     [+(           ] AB
-     [+(-          ] AB
C     [+(-          ] ABC
)     [+            ] ABC-
*     [+*           ] ABC-
D     [+*           ] ABC-D
/     [+/           ] ABC-D*
E     [+/           ] ABC-D*E
end   [             ] ABC-D*E/+
Postfix: ABC-D*E/+


Infix: A^B^C
read  stack         output
A     [             ] A
^     [^            ] A
B     [^            ] AB
^     [^^           ] AB
C     [^^           ] ABC
end   [             ] ABC^^
Postfix: ABC^^
```

```
Infix: (A+B
read  stack         output
(    [(          ]
A    [(          ] A
+    [(+         ] A
B    [(+         ] AB
Postfix: error (unbalanced brackets)
```

Stack contents after each character (derived by hand first, then confirmed by the run):

Formula 1: A*B+C			**Formula 2: A+(B-C)*D/E**		
Read	**Stack**	**Output**	**Read**	**Stack**	**Output**
A		A	A		A
*	*	A	+	+	A
B	*	AB	(	+ (	A
+	+	AB*	B	+ (	AB
C	+	AB*C	-	+ (-	AB
end		AB*C+	C	+ (-	ABC
			)	+	ABC-
			*	+ *	ABC-
			D	+ *	ABC-D
			/	+ /	ABC-D*
			E	+ /	ABC-D*E
			end		ABC-D*E/+

In formula 1, * has higher precedence than +, so when + arrives the * is popped to the output first and the + waits on the stack until the end. In formula 2 the) empties the bracket (- goes to the

output) and discards (; then / arrives while * is on top, and because * and / have equal precedence and are left-associative, the * is popped before / is pushed. The + stays at the bottom of the stack and is output last.

Two more cases are in the output: A^B^C gives ABC^^ because ^ is right-associative (an equal-precedence ^ already on the stack is **not** popped), and (A+B is rejected as unbalanced because a (is still on the stack when the formula ends.

3.5 Inquiry: time, space, overflow, underflow

Measure	Result for a formula of length n
Time (bracket check and conversion)	O(n): each character is read once and pushed at most once and popped at most once
Space	O(n) worst case, for a formula such as ((((((where every character is pushed; O(depth) for typical formulas

Overflow is a push on a full stack; underflow is a pop on an empty stack. Without checks, overflow writes past the end of the array and underflow reads item[-1], both undefined behaviour. In this program balanced rejects the formula as soon as the depth would pass MAX − 1, and pop returns '\0', which balanced treats as an underflow. Program q3_stack_edge.c shows both events with a stack of capacity 3.

```
/* Task 3 - Overflow and underflow: array stack (capacity 3) and linked stack */
#include <stdio.h>
#include <stdlib.h>


#define CAP 3
typedef struct { int item[CAP]; int top; } AStack;


static int apush(AStack *s, int v) { if (s->top == CAP - 1) return 0; s->item[++s->top] = v; return 1; }
static int apop(AStack *s, int *v) { if (s->top < 0) return 0; *v = s->item[s->top--]; return 1; }


typedef struct LNode { int v; struct LNode *next; } LNode;
typedef struct CNode { char v; struct CNode *next; } CNode;   /* char payload, for the size comparison */
static int lpush(LNode **h, int v)
{
   LNode *n = malloc(sizeof *n);
   if (!n) return 0;
```

```
    n->v = v; n->next = *h; *h = n;
    return 1;
}
static int lpop(LNode **h, int *v)
{
    if (!*h) return 0;
    LNode *n = *h;
    *v = n->v; *h = n->next;
    free(n);
    return 1;
}


int main(void)
{
    AStack a = {.top = -1};
    int v;
    for (int i = 1; i <= 4; i++)
        printf("array  push(%d) -> %s\n", i, apush(&a, i) ? "ok" : "OVERFLOW (rejected)");
    for (int i = 0; i < 4; i++) {
        if (apop(&a, &v)) printf("array  pop -> %d\n", v);
        else printf("array  pop -> UNDERFLOW (rejected)\n");
    }


    LNode *l = NULL;
    for (int i = 1; i <= 4; i++)
        printf("linked push(%d) -> %s\n", i, lpush(&l, i) ? "ok" : "failed");
    for (int i = 0; i < 5; i++) {
        if (lpop(&l, &v)) printf("linked pop -> %d\n", v);
        else printf("linked pop -> UNDERFLOW (rejected)\n");
    }
    printf("sizeof(char)=%zu  sizeof(CNode with char payload)=%zu\n", sizeof(char), sizeof(CNode));
    return 0;
}
```

Actual output (captured in terminal):

```
array  push(1) -> ok
array  push(2) -> ok
array  push(3) -> ok
array  push(4) -> OVERFLOW (rejected)
array  pop -> 3
array  pop -> 2
array  pop -> 1
array  pop -> UNDERFLOW (rejected)
linked push(1) -> ok
```

```
linked push(2) -> ok
linked push(3) -> ok
linked push(4) -> ok
linked pop -> 4
linked pop -> 3
linked pop -> 2
linked pop -> 1
linked pop -> UNDERFLOW (rejected)
sizeof(char)=1  sizeof(CNode with char payload)=16
```

3.6 Alternative Concept: linked-list stack

Push and pop work at the head of a singly linked list; top is the head pointer (the full working version is in q3_stack_edge.c above).

```
typedef struct LNode { int v; struct LNode *next; } LNode;
int lpush(LNode **h, int v) {                  /* O(1) */
   LNode *n = malloc(sizeof *n); if (!n) return 0;
   n->v = v; n->next = *h; *h = n; return 1;
}
int lpop(LNode **h, int *v) {                  /* O(1) */
   if (!*h) return 0;
   LNode *n = *h; *v = n->v; *h = n->next; free(n); return 1;
}
```

Measure	Array stack	Linked stack
push / pop time	O(1)	O(1), plus a malloc / free per operation
Space per element	1 byte (char), but the whole capacity is reserved	16 bytes for a char payload (measured sizeof), allocated only as needed
Very long formula	rejected once depth passes MAX (100 in this program)	grows until memory runs out; no fixed limit
Cache behaviour	contiguous	scattered nodes

Measured (q3_bench.c, -O2, best of 3, time in ms to push n characters and pop them all):

```
sizeof(LNode)=16 bytes for a char payload
        n     array (ms)    linked (ms)    ratio
  1000000          1.342         23.988    17.9x
  5000000          6.293        114.067    18.1x
```

10000000 11.921 239.744 20.1x

n	Array stack (ms)	Linked stack (ms)	Ratio
1,000,000	1.342	23.988	17.9×
5,000,000	6.293	114.067	18.1×
10,000,000	11.921	239.744	20.1×

Both grow linearly with n (O(1) per operation), but the linked stack is about 18 to 20 times slower per operation because of malloc and free and poor cache use. The linked stack is the right choice only when the depth is unknown and could exceed any sensible fixed array.

3.7 Viva questions with model answers

1. **Why must the stack be empty at the end of the bracket check?** A leftover opener such as the first (in =((B2+C2)*D2 has no closing bracket.
2. **Why does =SUM(A1:A5)) fail?** The second) is read while the stack is empty (underflow): there is no opener to match.
3. **Why is ^ handled differently in conversion?** It is right-associative, so an equal-precedence ^ already on the stack must not be popped.
4. **Which is faster, array or linked stack?** Both are O(1) per operation; the array is faster in practice (measured about 18 to 20 times).

3.8 Conclusion

The array stack checks brackets and converts infix to postfix in O(n) time and O(n) worst-case space; the traces show the mismatch in =[A1+(B2*C3]) at the] after C3. Overflow and underflow must be handled by returning an error, never by writing outside the array. A linked stack removes the depth limit at a measured cost of about 20 times more time per operation.

Q4 – Queues **(6 Marks)**

Aim. To explain why a linear array queue reports "queue full" while almost empty, to implement a circular queue with the modulo operator, to test wrap-around, to extend it for senior-citizen priority, and to compare it with a linked-list queue.

4.1 Why the linear queue fails

A linear queue enqueues at index rear and dequeues at index front, and both indexes only move forward. Cells before front are free but are never used again, because the full test is rear == SIZE. The first part of program q4_extra.c (SIZE = 4, one person waiting at a time) shows it. **Actual output (captured in terminal):**

```
(1) Linear queue, SIZE = 4, one person waiting at a time
  token 1: enqueued  front=0 rear=1 waiting=1
    served 1
  token 2: enqueued  front=1 rear=2 waiting=1
    served 2
  token 3: enqueued  front=2 rear=3 waiting=1
    served 3
  token 4: enqueued  front=3 rear=4 waiting=1
    served 4
  token 5: QUEUE FULL (false alarm)  front=4 rear=4 waiting=0
  token 6: QUEUE FULL (false alarm)  front=4 rear=4 waiting=0
```

After only four applicants the queue holds nobody (waiting=0) yet refuses token 5. With a 500-cell array the same thing happens after 500 applicants in total, however few are waiting. The fix is to reuse the freed cells by wrapping the indexes round the end of the array.

4.2 Design decisions for the circular queue

Question	Answer
Next free position	rear = (front + count) % SIZE
Move front after a dequeue	front = (front + 1) % SIZE
Why modulo	When the index would reach SIZE, % SIZE turns it back to 0, so the array behaves like a ring

Question	Answer
Tell full from empty	With front and rear alone, the empty and full states look the same (front == rear). Keep a count: empty is count == 0, full is count == SIZE. The alternative is to leave one cell always empty (full when (rear + 1) % SIZE == front), which wastes one cell but needs no counter.
When the queue is full	enqueue changes nothing and returns 0; the caller shows "queue full", asks the applicant to wait, or rejects the token. The program must never overwrite the front token.
Senior citizens	Keep a second circular queue for seniors and serve it first (with a fairness limit, section 4.5), or use a deque (double-ended queue) where seniors are inserted at the front.

Method for full / empty	Extra data	Usable cells	Test cost
Count variable	one int	SIZE	O(1)
One empty cell	none	SIZE − 1	O(1)
Flag bit	one flag	SIZE	O(1)

4.3 Complete the Code

Program q4_queue.c (SIZE = 5):

```
/* Task 4 - Circular queue for the passport-office token system (SIZE = 5) */
#include <stdio.h>


#define SIZE 5


typedef struct {
   int token[SIZE];
   int front, count;
} Queue;


int enqueue(Queue *q, int t)
{
   if (q->count == SIZE) return 0;                  /* full: reject */
   q->token[(q->front + q->count) % SIZE] = t;       /* next free cell */
```

```
    q->count++;
    return 1;
}


int dequeue(Queue *q, int *t)
{
    if (q->count == 0) return 0;                    /* empty */
    *t = q->token[q->front];
    q->front = (q->front + 1) % SIZE;               /* advance circularly */
    q->count--;
    return 1;
}


int main(void)
{
    Queue q = {.front = 0, .count = 0};
    int t;
    for (int i = 301; i <= 306; i++)
        if (!enqueue(&q, i)) printf("token %d: queue full\n", i);
    dequeue(&q, &t); printf("served %d\n", t);
    dequeue(&q, &t); printf("served %d\n", t);
    enqueue(&q, 307);                               /* wraps to cell 0 */
    enqueue(&q, 308);                               /* wraps to cell 1 */
    while (dequeue(&q, &t)) printf("served %d\n", t);
    printf("front index = %d\n", q.front);
    return 0;
}
```

Actual output (captured in terminal, compiled with -Wall -Wextra -pedantic, no warnings):

```
token 306: queue full
served 301
served 302
served 303
served 304
served 305
served 307
served 308
front index = 2
```

The output matches the hand-made dry run below: token 306 is refused, 301 and 302 are served, 307 and 308 wrap round into the freed cells, and the remaining tokens come out in arrival order.

Dry run (SIZE = 5; array shown as [0] [1] [2] [3] [4], . = unused):

Step	front	count	Array	Note
enqueue 301 … 305	0	5	301 302 303 304 305	cells 0 to 4
enqueue 306	0	5	301 302 303 304 305	count == SIZE: queue full, returns 0
dequeue → 301	1	4	. 302 303 304 305	front = (0 + 1) % 5
dequeue → 302	2	3	. . 303 304 305	front = (1 + 1) % 5
enqueue 307	2	4	307 . 303 304 305	index (2 + 3) % 5 = 0, wrap-around
enqueue 308	2	5	307 308 303 304 305	index (2 + 4) % 5 = 1
dequeue → 303	3	4	307 308 . 304 305	
dequeue → 304	4	3	307 308 . . 305	
dequeue → 305	0	2	307 308 . . .	front = (4 + 1) % 5 = 0, wrap
dequeue → 307	1	1	. 308 . . .	
dequeue → 308	2	0	. . . . .	queue empty, loop ends

Why the final front index is 2. Seven tokens were dequeued in all (301, 302, 303, 304, 305, 307, 308), and each dequeue moves front forward by one place modulo 5, so the final front is 7 mod 5 = 2. After the last enqueue (308) the array is [307, 308, 303, 304, 305] with front = 2 and count = 5; read from the front the queue order is 303, 304, 305, 307, 308.

```
index:  [0]  [1]  [2]  [3]  [4]
token:  307  308  303  304  305
                ^front=2     next free = (2 + 5) % 5 = 2  -> equals front, queue full (count = 5)
```

4.4 Test Your Design: wrap-around

The second part of q4_extra.c enqueues 17 tokens with at most 3 waiting, so the array of 4 cells is wrapped round more than four times, and checks that every token comes out in the order it went in. **Actual output (captured in terminal):**

```
(2) Circular queue, SIZE = 4, 17 tokens, at most 3 waiting
  served  1 (front 0 -> 1, count 2)
  served  2 (front 1 -> 2, count 2)
  served  3 (front 2 -> 3, count 2)
  served  4 (front 3 -> 0, count 2)
  served  5 (front 0 -> 1, count 2)
  served  6 (front 1 -> 2, count 2)
  served  7 (front 2 -> 3, count 2)
  served  8 (front 3 -> 0, count 2)
  served  9 (front 0 -> 1, count 2)
  served 10 (front 1 -> 2, count 2)
  served 11 (front 2 -> 3, count 2)
  served 12 (front 3 -> 0, count 2)
  served 13 (front 0 -> 1, count 2)
  served 14 (front 1 -> 2, count 2)
  served 15 (front 2 -> 3, count 2)
  served 16 (front 3 -> 0, count 1)
  served 17 (front 0 -> 1, count 0)
  tokens served = 17, order errors = 0, front wrapped from 3 to 0 on 4 occasions
  dequeue on empty queue returns 0
  enqueue(99) on a full queue returns 0, contents unchanged = yes
```

Test	Input	Expected	Actual
Dequeue from an empty queue	dequeue on a new queue	returns 0	0
Enqueue into a full queue	4 enqueues, then enqueue(99)	returns 0, queue unchanged	0, unchanged
Order after wrap-around	17 tokens through 4 cells	1, 2, …, 17 in order	0 order errors
Wrap-around of front	17 dequeues	front returns from 3 to 0	4 times

4.5 Extension: priority for senior citizens

Two circular queues are used: one for seniors and one for everyone else. The counter serves seniors first, but after 3 seniors in a row one general token is served, so general applicants cannot wait forever. The third part of q4_extra.c, with general tokens 401, 402, 403 and senior tokens 801 to 805, gives (**actual output, captured in terminal**):

```
(3) Priority service (fairness limit = 3 seniors in a row)
  serve senior  token 801
  serve senior  token 802
  serve senior  token 803
  serve general token 401
  serve senior  token 804
  serve senior  token 805
  serve general token 402
  serve general token 403
```

A deque gives the same effect with one array: general applicants join at the rear, seniors join at the front. But a senior inserted at the front jumps ahead of earlier seniors, so first-come-first-served among seniors is lost. Two queues keep the order inside each class, which is why they are preferred here.

4.6 Alternative Concept: linked-list queue

Keep front and rear pointers. Enqueue adds a node at rear; dequeue removes the node at front. The fourth part of q4_extra.c runs it (**actual output, captured in terminal**):

```
(4) Linked queue
  served: 301 302 303 304 305 306 307 308
  mallocs = 8, frees = 8, sizeof(QN) = 16 bytes (array cell = 4 bytes)
```

```
typedef struct QN { int v; struct QN *next; } QN;
typedef struct { QN *front, *rear; } LKQ;
void kenq(LKQ *q, int v) {                    /* O(1) */
    QN *n = malloc(sizeof *n); n->v = v; n->next = NULL;
    if (q->rear) q->rear->next = n; else q->front = n;
    q->rear = n;
}
int kdeq(LKQ *q, int *v) {                    /* O(1) */
    if (!q->front) return 0;
    QN *n = q->front; *v = n->v; q->front = n->next;
    if (!q->front) q->rear = NULL;             /* queue became empty */
```

```
    free(n); return 1;
}
```

Tokens 301 to 308 were served in order with mallocs = 8 and frees = 8 (AddressSanitizer clean), and sizeof(QN) = 16 bytes against 4 bytes per token in the array.

Measure	Circular array	Linked queue
Enqueue / dequeue time	O(1)	O(1)
Space per token	4 bytes (+ unused cells; total fixed at SIZE × 4)	16 bytes, allocated as needed
Capacity	fixed ("queue full" possible)	limited only by memory
Per-applicant cost	none	one malloc and one free

Measured (q4_bench.c, -O2, best of 3, about 100 applicants waiting, time in ms for n enqueue + dequeue pairs):

```
      n    circular (ms)    linked (ms)    ratio
1000000           2.961          9.449    3.2x
10000000         29.988         94.865    3.2x
```

n	Circular array (ms)	Linked queue (ms)	Ratio
1,000,000	2.961	9.449	3.2×
10,000,000	29.988	94.865	3.2×

Both scale linearly (O(1) per operation); the linked queue is about 3 times slower because of the malloc / free for every applicant. The array wins when a safe upper bound on waiting applicants is known; the linked queue wins when the bound is unknown.

4.7 Viva questions with model answers

1. **Why does front == rear not identify an empty queue?** In a circular queue a full queue also has front == rear; a count (or an always-empty cell) separates the two states.

2. **Why use (front + count) % SIZE?** It is the first free cell after the last token, wrapped round the end of the array.
3. **What happens to the token that finds the queue full?** It is rejected and reported; the stored tokens are not touched.
4. **Is the order of tokens preserved after wrap-around?** Yes; the test with 17 tokens through 4 cells gave 0 order errors.

4.8 Conclusion

The linear queue fails because freed front cells are never reused. The circular queue reuses them with modulo arithmetic, keeps FIFO order through every wrap-around, and tells full from empty with a count. Priority service is added with a second queue and a fairness limit. A linked queue removes the capacity limit at the cost of about 3 times more time per operation in the measurement.

Q5 – Linear Search versus Binary Search (7 Marks)

Aim. To implement linear and binary search on register numbers, to derive the best, average and worst case comparison counts, to measure the growth for n = 1,000, 100,000 and 300,000, to find after how many look-ups sorting pays for itself, and to compare with hashing and interpolation search.

5.1 Best, average and worst case

Search	Best case	Average case	Worst case	Requires
Linear	1 comparison: key is the first element	successful: (n + 1) / 2; unsuccessful: n	n comparisons: key is last, or absent	nothing (unsorted data is fine)
Binary	1 comparison: key is exactly the first middle element	successful: about $\log_2 n - 1$; unsuccessful: about $\log_2 n$	$\lfloor \log_2 n \rfloor + 1$ comparisons: key absent, or found at the last step	sorted data with O(1) random access

Average of linear search. If the key is equally likely to be at any of the n positions, position i costs i comparisons, so the average is $(1 + 2 + \ldots + n) / n = (n + 1) / 2$.

Binary search. Each comparison halves the interval, so after k comparisons at most $n / 2^k$ elements remain; the interval becomes empty when $2^k > n$, giving at most $\lfloor \log_2 n \rfloor + 1$ comparisons. For n = 300,000, $\log_2 n = 18.19$, so at most 19 comparisons, against 300,000 for linear search.

5.2 Predict the Output

Program q5_predict.c (12 sorted register numbers, keys 52 and 20):

```
/* Task 5 - Predict the Output: binary search trace on 12 sorted register numbers */
#include <stdio.h>


int main(void)
{
   int a[12] = {4, 9, 17, 23, 31, 38, 44, 52, 60, 71, 85, 93};
   int keys[2] = {52, 20};


   for (int k = 0; k < 2; k++) {
      int lo = 0, hi = 11, found = -1;
      while (lo <= hi) {
```

```
        int mid = (lo + hi) / 2;
        printf("%d ", a[mid]);
        if (a[mid] == keys[k]) { found = mid; break; }
        if (a[mid] < keys[k]) lo = mid + 1; else hi = mid - 1;
      }
      printf("-> %d\n", found);
    }
    return 0;
}
```

My predicted output (derived by hand before running):

```
38 60 44 52 -> 7
38 17 23 -> -1
```

Key	Step	lo	hi	mid	a[mid]	Decision
52	1	0	11	5	38	38 < 52: lo = 6
52	2	6	11	8	60	60 > 52: hi = 7
52	3	6	7	6	44	44 < 52: lo = 7
52	4	7	7	7	52	found at index 7
20	1	0	11	5	38	38 > 20: hi = 4
20	2	0	4	2	17	17 < 20: lo = 3
20	3	3	4	3	23	23 > 20: hi = 2
20	end	3	2	–	–	lo > hi: not found, result −1

Actual output (captured in terminal):

```
38 60 44 52 -> 7
38 17 23 -> -1
```

The actual output equals the prediction. The unsuccessful search for 20 needed 3 comparisons, which is within the worst case $\lfloor \log_2 12 \rfloor + 1 = 4$; a linear search for 20 would need all 12.

5.3 Implementation and the overflow of (lo + hi) / 2

Program q5_search.c returns the index and the number of comparisons, and tests the first, a middle, the last and an absent key:

```
/* Task 5 - Linear search and binary search, with comparison counting,
   plus a demonstration of why (lo + hi) / 2 can overflow. */
#include <stdio.h>


int linear(const int a[], int n, int key, int *cmp)
{
    *cmp = 0;
    for (int i = 0; i < n; i++) {
        ++*cmp;
        if (a[i] == key) return i;
    }
    return -1;
}


int binary(const int a[], int n, int key, int *cmp)
{
    int lo = 0, hi = n - 1;
    *cmp = 0;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;       /* overflow-safe midpoint */
        ++*cmp;
        if (a[mid] == key) return mid;
        if (a[mid] < key) lo = mid + 1; else hi = mid - 1;
    }
    return -1;
}


int main(void)
{
    int a[] = {4, 9, 17, 23, 31, 38, 44, 52, 60, 71, 85, 93};
    int n = 12;
    int keys[] = {4, 44, 93, 20};           /* first, middle, last, absent */
    for (int i = 0; i < 4; i++) {
        int c1, c2;
        int r1 = linear(a, n, keys[i], &c1), r2 = binary(a, n, keys[i], &c2);
        printf("key=%2d  linear=%2d (%2d comparisons)  binary=%2d (%d comparisons)\n",
               keys[i], r1, c1, r2, c2);
    }


    /* Overflow demonstration, done in unsigned arithmetic so the program itself is safe */
```

```
    int lo = 1500000000, hi = 2000000000;
    int wrapped = (int)((unsigned)lo + (unsigned)hi);      /* what the CPU computes */
    printf("lo=%d hi=%d  (lo+hi)/2 -> %d   lo+(hi-lo)/2 -> %d\n",
           lo, hi, wrapped / 2, lo + (hi - lo) / 2);
    return 0;
}
```

Actual output (captured in terminal, no warnings):

```
key= 4  linear= 0 ( 1 comparisons)  binary= 0 (3 comparisons)
key=44  linear= 6 ( 7 comparisons)  binary= 6 (3 comparisons)
key=93  linear=11 (12 comparisons)  binary=11 (4 comparisons)
key=20  linear=-1 (12 comparisons)  binary=-1 (3 comparisons)
lo=1500000000 hi=2000000000  (lo+hi)/2 -> -397483648   lo+(hi-lo)/2 -> 1750000000
```

- The first key costs 1 comparison for linear search but 3 for binary search: linear search wins when the key is at the very front, binary search wins on average. The last key (93) shows the binary worst case of $4 = \lfloor \log_2 12 \rfloor + 1$.
- **Why mid = lo + (hi - lo) / 2.** lo + hi can exceed the largest int (2,147,483,647) for a very large array; the sum 3,500,000,000 then wraps to −794,967,296 (half of it is −397,483,648) and a[mid] reads outside the array. hi - lo never exceeds hi, so lo + (hi - lo) / 2 stays inside the valid range. (The program uses unsigned arithmetic only to show the wrapped value safely; signed overflow itself is undefined behaviour in C.)
- **Why binary search is not efficient on a linked list.** The algorithm needs a[mid] in O(1). A list must walk to the middle node: about $n/2 + n/4 + n/8 + \ldots \approx n$ pointer moves in total, so the search is O(n) in time even though it makes only $\log_2 n$ comparisons. A sorted array, a binary search tree or a skip list is needed.

5.4 Growth investigation (measured)

Program q5_growth.c builds n distinct register numbers in random order, then makes 1,000 random look-ups (500 present, 500 absent) with each method. Compiled with gcc -std=c11 -O2; fixed random seed; each timing is the best of 3 runs. **Actual output (captured in terminal):**

```
     n |  lin avg cmp      0.75 n    lin ms |  bin cmp   log2 n    bin ms |  qsort ms
  1000 |       756.0       750.0     0.503 |     9.45     9.97    0.0358 |     0.062
       hash: 0.68 node visits/look-up, 0.0014 ms, build 0.007 ms, 2048 buckets, load 0.49 | interpolation: 1.33 probes, 0.0050 ms
       break-even by time: 133 look-ups | by comparisons: 13.3 look-ups | memory: array 4 KB, hash 32 KB
```

```
100000 |     73878.9     75000.0   47.712 |    16.15    16.61    0.0947 |    13.293
      hash: 0.65 node visits/look-up, 0.0015 ms, build 2.754 ms, 262144 buckets, load 0.38 | interpolation: 1.33 probes, 0.0051 ms
      break-even by time: 279 look-ups | by comparisons: 22.5 look-ups | memory: array 391 KB, hash 3610 KB

300000 |    224650.2    225000.0  147.647 |    17.75    18.19    0.1105 |    44.510
      hash: 0.88 node visits/look-up, 0.0019 ms, build 6.789 ms, 524288 buckets, load 0.57 | interpolation: 1.32 probes, 0.0055 ms
      break-even by time: 302 look-ups | by comparisons: 24.3 look-ups | memory: array 1172 KB, hash 8784 KB

Interpolation on SKEWED keys (n=100000): 22659.0 probes per look-up (binary would need about 16.6)
```

Linear and binary search against theory (times are for all 1,000 look-ups):

n	Linear: average comparisons	Theory for this mix: 0.75 n	Linear time (ms)	Binary: average comparisons	$\log_2 n$	Binary time (ms)
1,000	756.0	750	0.503	9.45	9.97	0.0358
100,000	73,878.9	75,000	47.712	16.15	16.61	0.0947
300,000	224,650.2	225,000	147.647	17.75	18.19	0.1105

The linear average matches the theory: half of the look-ups are present keys (about n/2 comparisons) and half are absent keys (n comparisons), so the mix averages 0.75 n. The binary average sits just below $\log_2 n$, as predicted. When n grows 300 times (1,000 to 300,000), the linear comparison count grows 297 times and the linear time about 294 times (O(n)), while the binary comparison count grows only 1.9 times and the binary time 3.1 times (O(log n)). At n = 300,000 binary search needs about 12,700 times fewer comparisons and is about 1,300 times faster by the clock. The time gap is smaller than the comparison gap because binary search jumps across a 1.2 MB array (cache misses) while the linear scan reads memory in order.

Cost of sorting once and the break-even point:

n	Sort once with qsort (ms)	Time saved per look-up (ms)	Break-even by measured time	Break-even by comparison count
1,000	0.062	0.000467	133 look-ups	$n \log_2 n / (756 - 9.45) = 13.3$

n	Sort once with qsort (ms)	Time saved per look-up (ms)	Break-even by measured time	Break-even by comparison count
100,000	13.293	0.047617	279 look-ups	1.66×10^6 / 73,863 = 22.5
300,000	44.510	0.147537	302 look-ups	5.46×10^6 / 224,632 = 24.3

Break-even = sorting cost divided by the saving per look-up. By comparison count, sorting (about $n \log_2 n$ comparisons) pays for itself after only 13 to 24 look-ups. By clock time the break-even is larger (133 to 302 look-ups) because one comparison inside qsort (a call through a function pointer) costs about 10 to 12 times more than one comparison in a simple scan loop. Either way, on result day with 300,000 look-ups the one-time sort is repaid within the first 0.1 % of the requests. Keeping the array sorted afterwards costs O(n) per insertion (shifting), which does not matter for a result list that is published once.

Measurement note: this container is a shared single-core machine, and repeated runs of the same program moved the sub-millisecond timings by 10 to 40 %. The comparison counts never changed, so the conclusions rest on them; replace the time columns with your own machine's numbers.

5.5 Alternative Concept: hashing and interpolation search

Hash table. Each register number is stored in bucket $h(key) = (key \times 2654435761) >> (32 - b)$, with chaining; the number of buckets $m = 2^b$ is the next power of two above n / 0.75. **Design note:** the shift keeps the *high* b bits of the product (Fibonacci hashing). My first version kept the low bits with & (m - 1), but the low bits of a product depend only on the low bits of the key, so keys that differ only in higher bits collided, and in my test every absent key fell into an always-empty bucket and looked free. Testing present and absent keys separately exposed this; the high-bit version behaves as theory predicts (present key ≈ 1 + load/2 node visits, absent key ≈ load factor).

Interpolation search guesses the position from the key value, assuming roughly uniform numbers: pos = lo + (key - a[lo]) * (hi - lo) / (a[hi] - a[lo]).

Measured (same look-ups as above):

n	Hash: node visits per look-up	Hash time (ms)	Hash build (ms), buckets, load factor	Interpolation: probes per look-up	Interpolation time (ms)
1,000	0.68	0.0014	0.007, 2,048, 0.49	1.33	0.0050
100,000	0.65	0.0015	2.754, 262,144, 0.38	1.33	0.0051
300,000	0.88	0.0019	6.789, 524,288, 0.57	1.32	0.0055

The hash averages below 1 visit because half of the look-ups are absent keys and many of them reach an empty bucket (0 visits); a present key needs 1 or slightly more. Interpolation needs only about 1.3 probes here because the test register numbers are nearly evenly spaced (4 apart with small random jitter), which is the best case for it. The last line of the program output shows the opposite case: for **skewed** keys (90 % evenly spaced, 10 % huge values) interpolation needed 22,659 probes per look-up, against about 16.6 for binary search, so it can degrade towards O(n).

Method	Time per look-up	Extra space (n = 300,000)	Needs sorted data	Weakness
Linear search	O(n)	none	no	slow growth
Binary search	O(log n) worst case	none (after in-place sort)	yes	sort cost; O(n) insertion
Interpolation search	O(log log n) average for uniform keys, O(n) worst	none	yes	needs near-uniform keys
Hash table	O(1) average, O(n) worst	about 8,784 KB (buckets + nodes) against 1,172 KB for the plain int array	no	extra memory; no sorted order; a poor hash function degrades it

5.6 Viva questions with model answers

1. **Why does binary search need sorted data?** It discards half the interval by comparing with the middle; that is only valid if everything left of the middle is smaller and everything right is larger.
2. **When is linear search the better choice?** For a small or unsorted array searched only a few times, and when the key is likely near the front.
3. **How many comparisons for 300,000 sorted records?** At most 19.
4. **Why is the (n + 1) / 2 average only for successful searches?** An unsuccessful search always inspects all n elements.

5.7 Conclusion

Linear search costs about n/2 to n comparisons and binary search about $\log_2 n$, confirmed by measurement at n = 300,000 (224,650 against 17.75 comparisons). Sorting once repays itself after 13 to 24 look-ups by comparison count (about 300 by clock time), and `mid = lo + (hi - lo) / 2` avoids integer overflow. Hashing is faster still but needs about 7.5 times the memory, and interpolation search is excellent for uniform keys but can collapse on skewed ones.

Q6 – Suffix Arrays and LCP Arrays **(8 Marks)**

Aim. To build the suffix array of a text, to compute the LCP array with Kasai's algorithm, to use both to find the longest repeated substring and to search for a pattern, to verify "banana" by hand, to detect text shared between two answer scripts, and to measure the growth against brute force.

6.1 Theory

Term	Meaning
Suffix i	the part of the text from position i to the end
Suffix array sa	the starting positions of all suffixes, listed in the alphabetical order of the suffixes
Rank array rank	the inverse of sa: rank[sa[r]] = r, the place of suffix i in the sorted list
LCP array lcp	lcp[r] = length of the longest common prefix of the suffixes sa[r−1] and sa[r]; lcp[0] = 0

Why the longest repeated substring lies between neighbouring suffixes. Suppose a substring s occurs at positions i and j. Then suffixes i and j both start with s. In alphabetical order, every suffix that lies between two strings with the same prefix s must also start with s. So all suffixes beginning with s form one block of consecutive entries in sa, and two neighbours inside that block share at least |s| characters. Hence the largest value in the LCP array is exactly the length of the longest repeated substring, and it is found without comparing all pairs.

6.2 Correctness check: "banana" by hand

The suffixes in text order, and in sorted order:

Position	Suffix	Rank r	sa[r]	Sorted suffix	lcp[r] (with the suffix above)
0	banana	0	5	a	0 (no suffix above)
1	anana	1	3	ana	1 (a)
2	nana	2	1	anana	3 (ana)
3	ana	3	0	banana	0
4	na	4	4	na	0

Position	Suffix	Rank r	sa[r]	Sorted suffix	lcp[r] (with the suffix above)
5	a	5	2	nana	2 (na)

Suffix array = [5, 3, 1, 0, 4, 2]; LCP array = [0, 1, 3, 0, 0, 2]. The largest LCP is 3 at rank 2, between ana and anana, so the longest repeated substring is **ana** (length 3, occurring at positions 1 and 3, overlapping). The program q6_tools.c confirms it; its first section prints:

```
== banana ==
 r  sa  lcp  suffix
 0   5    0  a
 1   3    1  ana
 2   1    3  anana
 3   0    0  banana
 4   4    0  na
 5   2    2  nana
longest repeated substring: ana (length 3)
pattern 'ana' occurs at: 3 1  (2 occurrences)
pattern 'na' occurs at: 4 2  (2 occurrences)
pattern 'nan' occurs at: 2  (1 occurrence)
pattern 'xyz' occurs at:  (0 occurrences)
```

The program output equals the hand-made arrays. (The last two pattern searches are explained in section 6.4.)

6.3 Complete the Code: Kasai's algorithm

Program q6_kasai.c. The loop body takes j, the suffix just before suffix i in sorted order; extends h while the characters match; stores lcp[rank[i]] = h; then decreases h by 1 if it is positive. The test text is a short DNA read, GATTACAGATTACA, with the repeat GATTACA.

```
/* Task 6 - Kasai's algorithm on a short DNA read, with a trace of every iteration */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>


static const char *text;


int cmp(const void *a, const void *b)
{
   return strcmp(text + *(const int *)a, text + *(const int *)b);
```

```
}


int main(void)
{
    text = "GATTACAGATTACA";                /* a short DNA read with one repeat */
    int n = (int)strlen(text);
    int sa[64], rank[64], lcp[64] = {0};


    for (int i = 0; i < n; i++) sa[i] = i;
    qsort(sa, (size_t)n, sizeof sa[0], cmp);            /* suffix array, simple O(n^2 log n) */
    for (int i = 0; i < n; i++) rank[sa[i]] = i;        /* inverse of sa */


    printf("Trace of Kasai's loop (h is carried from one row to the next)\n");
    printf("%2s %5s %3s %6s %8s %8s %6s\n", "i", "rank", "j", "h_in", "h_after", "lcp[rk]", "h--");


    /* Kasai: lcp[r] = longest common prefix of suffixes sa[r-1] and sa[r] */
    int h = 0;
    for (int i = 0; i < n; i++) {
        if (rank[i] == 0) {                         /* no suffix above it */
            printf("%2d %5d %3s %6d %8s %8s %6s\n", i, rank[i], "-", h, "-", "skipped", "0");
            h = 0;
            continue;
        }
        int j = sa[rank[i] - 1];                    /* suffix just before suffix i */
        int h_in = h;
        while (i + h < n && j + h < n && text[i + h] == text[j + h]) h++;
        lcp[rank[i]] = h;
        int h_after = h;
        if (h > 0) h--;                             /* next suffix shares >= h-1 */
        printf("%2d %5d %3d %6d %8d %8d %6d\n", i, rank[i], j, h_in, h_after, lcp[rank[i]], h);
    }


    printf("\n%2s %3s %3s  %s\n", "r", "sa", "lcp", "suffix");
    int best = 0;
    for (int r = 0; r < n; r++) {
        printf("%2d %3d %3d  %s\n", r, sa[r], lcp[r], text + sa[r]);
        if (lcp[r] > lcp[best]) best = r;
    }
    printf("\nlongest repeat: %.*s (length %d)\n", lcp[best], text + sa[best], lcp[best]);
    return 0;
}
```

Actual output (captured in terminal, compiled with -Wall -Wextra -pedantic and with AddressSanitizer, no warnings or errors):

```
Trace of Kasai's loop (h is carried from one row to the next)
 i  rank   j   h_in  h_after  lcp[rk]   h--
 0    9   7     0       7       7     6
 1    5   8     6       6       6     5
 2   13   9     5       5       5     4
 3   11  10     4       4       4     3
 4    2  11     3       3       3     2
 5    7  12     2       2       2     1
 6    3   4     1       1       1     0
 7    8   5     0       0       0     0
 8    4   6     0       1       1     0
 9   12   3     0       1       1     0
10   10   0     0       0       0     0
11    1  13     0       1       1     0
12    6   1     0       0       0     0
13    0   -     0       -  skipped     0


 r  sa lcp  suffix
 0  13   0  A
 1  11   1  ACA
 2   4   3  ACAGATTACA
 3   6   1  AGATTACA
 4   8   1  ATTACA
 5   1   6  ATTACAGATTACA
 6  12   0  CA
 7   5   2  CAGATTACA
 8   7   0  GATTACA
 9   0   7  GATTACAGATTACA
10  10   0  TACA
11   3   4  TACAGATTACA
12   9   1  TTACA
13   2   5  TTACAGATTACA


longest repeat: GATTACA (length 7)
```

Here sa = [13, 11, 4, 6, 8, 1, 12, 5, 7, 0, 10, 3, 9, 2] and rank = [9, 5, 13, 11, 2, 7, 3, 8, 4, 12, 10, 1, 6, 0]. By hand: suffix 0 (GATTACAGATTACA) and suffix 7 (GATTACA) share 7 characters, so lcp[rank[0]] = lcp[9] = 7, and the longest repeat is GATTACA.

Trace of Kasai's loop (h is carried from one row to the next; taken from the program output above):

i	rank[i]	j = sa[rank[i] − 1]	h on entry	h after the while loop	stored lcp[rank[i]]	h after h--
0	9	7	0	7	7	6
1	5	8	6	6	6	5
2	13	9	5	5	5	4
3	11	10	4	4	4	3
4	2	11	3	3	3	2
5	7	12	2	2	2	1
6	3	4	1	1	1	0
7	8	5	0	0	0	0
8	4	6	0	1	1	0
9	12	3	0	1	1	0
10	10	0	0	0	0	0
11	1	13	0	1	1	0
12	6	1	0	0	0	0
13	0	–	0	–	skipped (no suffix above)	0

The LCP array by rank is [0, 1, 3, 1, 1, 6, 0, 2, 0, 7, 0, 4, 1, 5] (third column of the output). The largest value, 7 at rank 9, gives the repeat GATTACA. Notice the staircase 7, 6, 5, 4, 3, 2, 1 in the "h on entry" and "h after h--" columns for i = 0 to 6: the carried value h lets the while loop skip almost all comparisons, which is exactly why the algorithm is linear.

Why h can fall by only 1 at the next suffix. Let suffix j be the predecessor of suffix i in sorted order and let them share $h \geq 1$ characters. Remove the first character of both: suffix j+1 and suffix i+1 still share $h - 1$ characters, and j+1 is still alphabetically below i+1 (the removed first

characters were equal). The true predecessor of suffix i+1 is at least as close to it as suffix j+1, so it shares at least h − 1 characters with it. Therefore h can start the next step at h − 1 instead of 0.

Why this makes Kasai's algorithm O(n). h is never larger than n. It goes down at most once per suffix (by 1), so at most n times in total. It can therefore go up (by the `h++` inside the while loop) at most n + n = 2n times in total. The loop body costs O(1) outside the while loop, so the whole algorithm is O(n) time with O(n) extra space (the rank and lcp arrays).

6.4 Pattern search by binary search on the suffix array

All suffixes that start with a pattern P of length m form one block of consecutive entries in sa. Two binary searches (first suffix ≥ P, and first suffix after every suffix that starts with P) find the block. Each binary search makes O(log n) comparisons and each comparison looks at no more than m characters, so the search costs O(m log n), plus O(k) to list the k occurrences.

```
/* first rank whose suffix is >= P (upper = 0), or first rank after every suffix that starts with P (upper = 1) */
static int bound(const char *t, const int *sa, int n, const char *p, int m, int upper)
{
    int lo = 0, hi = n;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        int c = strncmp(t + sa[mid], p, (size_t)m);
        if (c < 0 || (upper && c == 0)) lo = mid + 1; else hi = mid;
    }
    return lo;
}


static void find_pattern(const char *t, const int *sa, int n, const char *p)
{
    int m = (int)strlen(p);
    int lo = bound(t, sa, n, p, m, 0), hi = bound(t, sa, n, p, m, 1);
    printf("pattern '%s' occurs at:", p);
    for (int r = lo; r < hi; r++) printf(" %d", sa[r]);
    printf("  (%d occurrence%s)\n", hi - lo, hi - lo == 1 ? "" : "s");
}
```

For "banana" the program found `ana` at positions 3 and 1, `na` at positions 4 and 2, `nan` at position 2 only, and reported 0 occurrences for the absent pattern `xyz` (see the output in section 6.2): the occurrences are exactly the entries between the two bounds, and an absent pattern gives an empty block.

6.5 Detecting text copied between two answer scripts

Join script A and script B with a separator that occurs in neither (here #): A#B. Build the suffix array and the LCP array of the joined text. A passage shared by both scripts appears as a pair of neighbouring suffixes, one starting in A and one in B; the longest such LCP is the longest shared passage. The # appears only once, so no common prefix can run across it.

```
for (int r = 1; r < N; r++) {
   int x = sa2[r - 1], y = sa2[r];
   int diff = (x < la) != (y < la) && x != la && y != la;   /* one suffix from each script */
   if (diff && lc2[r] > bl) { bl = lc2[r]; bp = y; }
}
```

For the test scripts "a queue is a fifo structure used for token systems in offices" and "students say a queue is a fifo structure used for printing jobs", the actual output (captured in terminal) was:

```
== shared passage ==
longest passage shared by both scripts (37 chars): "a queue is a fifo structure used for "
```

A long shared passage is evidence worth examining, not proof of copying: common definitions and quoted text also match, so a person must review the result.

6.6 Growth investigation (measured) and cost analysis

Program q6_tools.c builds the suffix array (qsort with strcmp) and the LCP array (Kasai) for random DNA text (alphabet A, C, G, T) and for the worst-case text of identical letters, and compares with the brute-force method that compares every pair of positions. gcc -O2, times in milliseconds. **Actual output (captured in terminal):**

```
== growth (ms) ==
     n text        suffix array     Kasai     SA+Kasai    brute force   repeat
  1000 random DNA        0.121     0.012        0.133          1.8       8 (match)
 10000 random DNA        1.873     0.154        2.026        238.7      13 (match)
 30000 random DNA        6.786     0.652        7.438       2013.9      13 (match)
100000 random DNA       23.966     1.738       25.704      21755.4      18 (match)
  1000 all A             0.086     0.003        0.089        132.3     999 (match)
  2000 all A             0.271     0.007        0.277       1066.3    1999 (match)
 10000 all A             6.506     0.032        6.537       not run   9999
```

n	Text	Suffix array (qsort + strcmp)	Kasai LCP	Total	Brute force (all pairs)	Longest repeat length
1,000	random DNA	0.121	0.012	0.133	1.8	8
10,000	random DNA	1.873	0.154	2.026	238.7	13
30,000	random DNA	6.786	0.652	7.438	2,013.9	13
100,000	random DNA	23.966	1.738	25.704	21,755.4	18
1,000	all A	0.086	0.003	0.089	132.3	999
2,000	all A	0.271	0.007	0.277	1,066.3	1,999
10,000	all A	6.506	0.032	6.537	not run	9,999

The brute-force repeat length equals the suffix-array result in every row where it was run (the program prints "match" for each). Reading the table:

- For random DNA, a 10-fold longer text makes brute force about 90 to 130 times slower (1.8 to 238.7 ms and 238.7 to 21,755 ms: $O(n^2)$ pairs), but the suffix array and Kasai only about 13 to 15 times slower (0.133 to 2.026 ms, then 2.026 to 25.704 ms: $O(n \log n)$). At $n = 100{,}000$ the suffix-array method is about 850 times faster (25.7 ms against 21.8 s).
- For the worst-case text, brute force at $n = 1{,}000$ takes 132.3 ms against 1.8 ms for random text (73 times more), because every pair matches almost to the end. Doubling n to 2,000 multiplies the time by 8.1 (132.3 to 1,066.3 ms), the $O(n^3)$ behaviour. It was not run for larger n.
- The suffix-array build for all-A text stays small only because the C library's strcmp is heavily optimised; the algorithm is still $O(n^2 \log n)$ in the worst case, since each comparison can inspect $O(n)$ characters. The growth is visible: 5 times more characters (2,000 to 10,000) cost 24 times more time (0.271 to 6.506 ms), close to quadratic.

6.7 Alternative Concept and Complexity Analysis

Method	Time	Extra space	Remarks
Brute force: compare every pair of starting positions	$O(n^2)$ pairs, each match up to $O(n)$: $O(n^3)$ worst case, $O(n^2 \times$ average repeat length) typically	$O(1)$	simple; measured 21.8 s at n = 100,000
Suffix array (qsort + strcmp) + Kasai LCP	sort $O(n \log n)$ comparisons $\times O(n)$ per comparison = $O(n^2 \log n)$ worst case, about $O(n \log n)$ for random text; Kasai $O(n)$	sa, rank, lcp: 3n ints (12n bytes) + the text	measured 25.7 ms at n = 100,000
Suffix array by prefix doubling	$O(n \log n)$ or $O(n \log^2 n)$	$O(n)$	removes the $O(n)$ strcmp factor
Suffix tree (Ukkonen)	$O(n)$ build; repeated substring by finding the deepest internal node, $O(n)$	$O(n)$ but with a large constant: up to about 2n nodes with child pointers	fastest in theory, much harder to implement

A suffix tree gives $O(n)$ time for everything but needs far more memory per character than the three integer arrays of the suffix array; the suffix array with Kasai is the practical middle: simple code, $O(n)$ extra space, and near-$O(n \log n)$ time.

6.8 Viva questions with model answers

1. **What does `lcp[r]` store?** The length of the longest common prefix of the suffixes at ranks r − 1 and r.
2. **Why can Kasai process suffixes in text order and not in sorted order?** Moving from suffix i to i + 1 changes the common prefix by at most 1, which is not true in sorted order.
3. **Why is `rank` needed?** To find, for a text position i, where its suffix sits in the sorted order, and so its predecessor `sa[rank[i] - 1]`.
4. **Why is the separator needed when joining two scripts?** Without it, a match could run from the end of A into the start of B and be reported as a shared passage.

6.9 Conclusion

The suffix array sorts all suffixes once, Kasai's algorithm fills the LCP array in O(n), and the largest LCP value is the longest repeated substring (ana for banana, GATTACA for GATTACAGATTACA). The method also gives O(m log n) pattern search and detection of shared passages between scripts. Measured at n = 100,000, it is about 850 times faster than comparing every pair of positions.

Q7 – Design Challenge: Diagnose, Redesign and Validate (13 Marks)

Chosen program: the linear search of Q5 (university result look-up). It is the program whose own evidence shows the clearest limitation as the data grows. All measurements in this question come from q7_redesign.c, which runs the original and both redesigns on the same records and the same 1,000 queries. Records are {register number, marks} (8 bytes) and the comparison counting is done through a pointer in all three searches, so absolute times differ from Q5 (4-byte integers, local counter). Only comparisons within one table are meaningful.

A. Diagnose the Limitation (2 Marks)

Evidence from the original program (first row of each group in the table of part E):

n	Average comparisons per look-up (500 present, 500 absent)	Time for 1,000 look-ups
1,000	760.98	0.289 ms
100,000	74,450.36	27.610 ms
300,000	223,031.31	85.424 ms

Diagnosis. The cost per look-up is proportional to n: when n grows 300 times (1,000 to 300,000) the time grows about 296 times and the comparisons about 293 times. A present key costs n/2 comparisons on average and an absent key costs all n, so the mix averages 0.75 n (225,000 expected, 223,031 measured). At 300,000 records one look-up takes about 0.085 ms, so the 300,000 look-ups of result day need about 26 seconds of pure search time (85.4 ms per 1,000 × 300), and the cost per look-up keeps growing with every new student.

Cause in the data structure. An unsorted array carries no information about where a key might be, so the only safe method is to inspect the elements one by one. The limitation is in the organisation of the data, not in the speed of the loop.

B. Define the Design Objective (2 Marks)

For n = 300,000 records and the same 1,000 queries (500 present, 500 absent, including the first and last record):

No.	Measurable objective
O1	Correctness: every query returns the same marks (or NOT FOUND) as the original; 0 disagreements
O2	Work: average comparisons per look-up at most 20 (original: 223,031)
O3	Speed: at least 100 times less time for the 1,000 look-ups than the original
O4	Preparation (sorting or building) is repaid within 1,000 look-ups, that is within 0.33 % of the 300,000 requests
O5	Total memory for the data structure at most 10 MB

The objectives follow from the diagnosis: the cost must stop growing in proportion to n (O2, O3) without breaking results (O1) or exceeding the memory and set-up budget (O4, O5).

C. Design the Revised Solution (4 Marks)

No library search routine is used; both redesigns are written by hand (only qsort is used to sort).

Redesign A: sort once, then binary search. The array of records is sorted by register number once (qsort). Each look-up halves the interval, so it needs at most $\lfloor \log_2 n \rfloor + 1 = 19$ comparisons for n = 300,000.

Redesign B: hash table with chaining. Each record is placed in bucket $h(key) = (key \times 2654435761) >> (32 - b)$, the high b bits of the product (Fibonacci hashing), where $m = 2^b$ is the first power of two not below n / 0.75 (524,288 buckets for 300,000 records, load factor 0.57). A look-up computes the bucket and walks its short chain.

```
/* ---------- ORIGINAL: linear search ---------- */
static int lin(const Rec *a, int n, int key, long *c)
{
   for (int i = 0; i < n; i++) { ++*c; if (a[i].reg == key) return a[i].marks; }
   return -1;
}
```

```
/* ---------- REDESIGN A: sorted array + binary search ---------- */
static int cmp_rec(const void *x, const void *y)
{
   int a = ((const Rec *)x)->reg, b = ((const Rec *)y)->reg;
   return (a > b) - (a < b);
}
static int bin(const Rec *a, int n, int key, long *c)
{
   int lo = 0, hi = n - 1;
   while (lo <= hi) {
      int mid = lo + (hi - lo) / 2;
      ++*c;
      if (a[mid].reg == key) return a[mid].marks;
      if (a[mid].reg < key) lo = mid + 1; else hi = mid - 1;
   }
   return -1;
}


/* ---------- REDESIGN B: hash table with chaining ---------- */
typedef struct HN { Rec r; struct HN *next; } HN;
typedef struct { HN **b; unsigned m; int bits; } Hash;
/* Fibonacci hashing: the HIGH "bits" bits of the product (low bits depend only on the low bits of the key) */
static unsigned hf(int k, int bits) { return ((unsigned)k * 2654435761u) >> (32 - bits); }
static void hput(Hash *h, Rec r)
{
   HN *n = malloc(sizeof *n);
   unsigned i = hf(r.reg, h->bits);
   n->r = r; n->next = h->b[i]; h->b[i] = n;
}
static int hget(const Hash *h, int key, long *c)
{
   HN *p = h->b[hf(key, h->bits)];
   if (!p) { ++*c; return -1; }                    /* empty bucket costs one probe */
   for (; p; p = p->next) { ++*c; if (p->r.reg == key) return p->r.marks; }
   return -1;
}
```

```
ORIGINAL     [r7][r2][r9][r1][r5] ...   unsorted: scan from the left, up to n steps
REDESIGN A   [r1][r2][r5][r7][r9] ...   sorted:   lo -- mid -- hi, halve the interval
REDESIGN B   bucket 0 -> r9 -> r2     bucket 1 -> (empty)     bucket 2 -> r7 ...   one bucket per look-up
```

Extra memory and new trade-offs:

Design	Memory for n = 300,000	Cost of keeping it current	Other trade-off
Original	2,344 KB	append O(1)	O(n) look-up
A: sorted array	2,344 KB (no extra array; qsort may use a temporary buffer while sorting)	insertion O(n) (records must shift)	needs a re-sort or an ordered insert after every change; gives ordered listing for free
B: hash table	8,784 KB (524,288 bucket pointers + 300,000 nodes)	insertion O(1)	3.7 times the memory; no sorted order; a poor hash function degrades chains toward O(n)

D. Predict Before Implementing (1 Mark)

Prediction, derived from the analysis and written before the redesigned searches were run (write your own before running your copy):

- **Redesign A:** at most 19 comparisons, average about 17 to 18 ($\log_2 300{,}000 = 18.2$), that is about 12,000 times fewer than 223,000. The gain is expected first in comparisons; the time ratio should be smaller than the comparison ratio, because each binary-search probe is a likely cache miss while the linear scan reads memory in order. Expected time reduction: several hundred to a few thousand times.
- **Redesign B:** about 1 to 1.5 comparisons (node visits or bucket probes) per look-up at load factor 0.57; time reduction larger than for A. Build cost O(n), a few tens of milliseconds at 300,000 records.
- **Update cost:** inserting records into the sorted array will be the slowest of the three (O(n) shifting); the hash insert and the original append will both be near-instant.

E. Implement and Validate the Redesign (2 Marks)

Both redesigns were run on the same data and the same 1,000 queries as the original (query 0 is the first record of the array, query 2 is the last record, other even queries are present keys drawn at random, odd queries are absent keys), with gcc 13.3.0, -O2. For the hash table, one comparison

means one key comparison, and an empty bucket counts as one probe. **Actual output (captured in terminal, AddressSanitizer run also clean):**

```
gcc -std=c11 -O2 | same 1000 queries (500 present, 500 absent) for all three designs
     n | design                         avg cmp      ms / 1000   build ms   memory KB
  1000 | original: linear search         760.98          0.289      0.000           8
       | A: sort + binary                 9.50          0.042      0.068           8
       | B: hash (chaining)               1.07          0.003      0.031          32
       | disagreements between the three designs over 1000 queries: 0
       | insert 1000 new records: unsorted array 0.002 ms, sorted array 0.127 ms, hash 0.036 ms


100000 | original: linear search       74450.36         27.610      0.000         781
       | A: sort + binary                16.15          0.098     13.019         781
       | B: hash (chaining)               1.05          0.003      6.173        3610
       | disagreements between the three designs over 1000 queries: 0
       | insert 1000 new records: unsorted array 0.005 ms, sorted array 9.198 ms, hash 0.187 ms


300000 | original: linear search      223031.31         85.424      0.000        2344
       | A: sort + binary                17.77          0.138     46.519        2344
       | B: hash (chaining)               1.14          0.004     26.066        8784
       | disagreements between the three designs over 1000 queries: 0
       | insert 1000 new records: unsorted array 0.004 ms, sorted array 30.399 ms, hash 0.120 ms
```

Original → redesigned, same tests (n = 300,000):

Measure	Original	A: sorted + binary	B: hash
Average comparisons per look-up	223,031.31	17.77 (12,550 times fewer)	1.14 (195,600 times fewer)
Time for 1,000 look-ups	85.424 ms	0.138 ms (619 times faster)	0.004 ms (over 15,000 times faster; at the 3-decimal limit of the timer)
Preparation	none	sort 46.519 ms	build 26.066 ms
Break-even (preparation ÷ time saved per look-up)	–	545 look-ups	305 look-ups
Memory	2,344 KB	2,344 KB	8,784 KB
Insert 1,000 new records	0.004 ms	30.399 ms	0.120 ms
Disagreements with the original	–	0	0

The speed-up grows with n: for time, A is 7 times faster at n = 1,000, 282 times at n = 100,000 and 619 times at n = 300,000, which is the O(n) against O(log n) gap opening up. For the hash table the gap is about 96, 9,200 and over 15,000 times. The prediction in part D held: A needed 17.77 comparisons (predicted 17 to 18), the time gain (619) lay inside the predicted several-hundred-to-thousands range and was far below the comparison gain, B was about 1.1 comparisons, and sorted insertion was the slowest update.

F. Defend the Final Design (2 Marks)

Were the objectives achieved at n = 300,000?

Objective	Result	Met?
O1 correctness	0 disagreements in 1,000 queries for both designs	Yes
O2 at most 20 comparisons	A: 17.77, B: 1.14	Yes
O3 at least 100 times faster	A: 619 times, B: over 15,000 times	Yes
O4 repaid within 1,000 look-ups	A: 545, B: 305	Yes
O5 at most 10 MB	A: 2.3 MB, B: 8.6 MB	Yes

Objective O3 is stated for n = 300,000 only; at n = 1,000 the gain is just 7 times, because both versions already finish in well under a microsecond per look-up.

Final design: sorted array with binary search for the result portal, because the results are published once and then only read. It meets every objective with no extra memory, keeps the data in register-number order for printing, and brings the total search time of the 300,000 requests from about 26 seconds (85.4 ms × 300) to about 41 ms (0.138 ms × 300). The hash table is about 30 times faster still (about 1 ms in total), but at 3.7 times the memory, with no order, and at a speed that no student would notice.

New costs of the redesign: a one-time sort of 46.5 ms; insertion into the sorted array costs O(n) (30.4 ms for 1,000 inserts, against 0.004 ms for the original); the hash table needs 3.7 times the memory and a good hash function. (My first hash used the low bits of the product and looked artificially fast on absent keys; switching to the high bits fixed it, which shows how much the hash function matters.)

Where the original may still be preferable:

- Small data (about 1,000 records or fewer): both finish in well under a microsecond per look-up, so the extra code gains nothing.
- Data that changes constantly and is searched rarely: fewer than about 545 look-ups between changes do not repay the sort, and the original appends in O(1).
- A one-off search of unsorted data that is read only once.

Choose the hash table instead if records are added or removed all day (insertion 0.120 ms against 30.4 ms for the sorted array at n = 300,000) and only exact-key look-up is needed.

The redesign succeeds because the measurements, not an estimate, show the objectives met. All timings are from one machine and one run each; repeat them on your own machine, report the median of several runs, and replace the tables if your numbers differ. The comparison counts will not change.